

□> **Templates** : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_1ère_année_FR_01.pdf

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS 1ère année Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques

Ce TD de synthèse permet de réviser les notions fondamentales du module avant d'aborder les séances approfondies. À l'issue de ce TD, l'étudiant est capable de :

- Situer les différents thèmes du module (architecture, OS, algorithmique, réseaux, sécurité, Python)
- Appliquer les concepts de base vus en introduction
- Établir des liens entre les différentes parties du programme

Contexte pédagogique et Scénario enseignant

Vous êtes enseignant en 1ère année ENS et devez faire une **évaluation diagnostique** de vos étudiants sur les fondamentaux de l'informatique. Ce TD synthétique couvre 4 thématiques clés en 90 min. Vous disposez d'une **grille d'évaluation** (page 2) pour mesurer le niveau initial de chaque étudiant. Utilisez les **pictogrammes** suivants pour guider la différenciation :

Symbole	Niveau	Attendu
☐	Débutant	Questions de base et tous les étudiants doivent répondre
☐...	Intermédiaire	Approfondissement et à traiter si le temps le permet
☐...☐...	Avancé	Défi pour les plus rapides, peut faire l'objet d'un bonus

Conseil d'animation : Corrigez collectivement après chaque exercice. Alternez temps individuel (5 min) et mise en commun (3 min). Pour l'exercice 5 (Python), projetez le script et faites-le exécuter en direct si possible.

Plan du TD

Phase	Durée	Total cumulé
Accueil	5 min	5 min
Exercice 1 et Architecture	15 min	20 min
Exercice 2 et Processus	18 min	38 min
Exercice 3 et Algorithme	18 min	56 min
Exercice 4 et Réseaux	18 min	74 min
Exercice 5 et Python	16 min	90 min

Total	90 min	
--------------	---------------	--

Sommaire conceptuel du module

```

mindmap
  root((Module Informatique))
    Architecture
      Von Neumann
      CPU & ALU
      Hiérarchie mémoire
    Systèmes d'exploitation
      Processus & threads
      Gestion mémoire
      Système de fichiers
    Algorithmique
      Variables & types
      Conditions & boucles
      Complexité
    Structures de données
      Tableaux
      Piles & files
      Listes chaînées
      Réseaux
      LAN / WAN
      IP / DNS
      Client-serveur
      Sécurité
      CID
      Mots de passe
      RGPD
      Python
      Recherche & tri
      Classes & objets
      Dictionnaires

```

Vue d'ensemble des exercices

```

graph LR
  subgraph "TD01 : Synthèse"
    A[Exercice 1<br/>Architecture] --> B[Exercice 2<br/>Processus]
    B --> C[Exercice 3<br/>Algorithmie]
    C --> D[Exercice 4<br/>Réseaux]
    D --> E[Exercice 5<br/>Python & Sécurité]
  end
  style A fill:#4a90d9,color:#fff
  style B fill:#2ecc71,color:#fff
  style C fill:#e67e22,color:#000
  style D fill:#e74c3c,color:#fff
  style E fill:#9b59b6,color:#fff

```

Exercice 1 : Architecture et codage (15 min)

Scénario enseignant : Vous devez vérifier que vos étudiants comprennent le rôle des composants fondamentaux d'un ordinateur. Cet exercice permet de sonder leurs connaissances sur l'architecture von Neumann.

Question de base (5 min)

Ã%noncÃ© : Expliquer le rÃ´le de l'ALU, de l'unitÃ© de contrÃ´le et de la mÃ©moire centrale. Convertir 13 en binaire.

CorrigÃ© dÃ©taillÃ© :

- **ALU** (Arithmetic Logic Unit) : effectue les opÃ©rations arithmÃ©tiques (addition, soustraction) et logiques (ET, OU, NON).
- **UC** (UnitÃ© de ContrÃ´le) : orchestre le cycle fetch-decode-execute, sÃ©quence le dÃ©roulement des instructions.
- **MÃ©moire centrale (RAM)** : stocke temporairement les instructions et les donnÃ©es en cours d'utilisation. Volatile.
- 13â„¤â„¤ = 1101â„¤â„¤, â„¤ mÃ©thode : divisions successives par 2 :

```
13 Ã· 2 = 6 reste 1 â†‘
6 Ã· 2 = 3 reste 0 |
3 Ã· 2 = 1 reste 1 |
1 Ã· 2 = 0 reste 1 |
Lecture des restes de bas en haut â†‘ 1101
```

Ã… Question intermÃ©diaire (5 min)

Ã%noncÃ© : Associer chaque composant Ã sa catÃ©gorie (entrÃ©e, sortie, traitement, contrÃ´le, mÃ©moire) :

Composant	RÃ´le	CatÃ©gorie
Clavier	Saisie de texte	?
Ã‰cran	Affichage visuel	?
ALU	Calculs arithmÃ©tiques	?
RAM	Stockage temporaire	?
Disque dur	Stockage permanent	?

CorrigÃ© : Clavier â†‘ EntrÃ©e, Ã‰cran â†‘ Sortie, ALU â†‘ Traitement, RAM â†‘ MÃ©moire centrale, Disque dur â†‘ MÃ©moire de masse (stockage permanent).

Ã…Ã… Question avancÃ©e (5 min)

Ã%noncÃ© : On vous donne le schÃ©ma simplifiÃ© suivant. ComplÃ©tez les flÃªches (nom du bus) et indiquez le sens de circulation des donnÃ©es.

```
graph LR
CPU["CPU<br/>(UC + ALU)"] -->|?| RAM["RAM"]
RAM -->|?| CPU
CPU -->|?| ENTREE["Clavier"]
CPU -->|?| SORTIE["Ã‰cran"]
HDD["Disque dur"] -->|?| RAM

style CPU fill:#4a90d9,color:#fff
style RAM fill:#2ecc71,color:#fff
style HDD fill:#e67e22,color:#000
```

CorrigÃ© :

- CPU â†‘ RAM : bus de donnÃ©es (bidirectionnel)
- CPU â†‘ Clavier : bus de contrÃ´le (lecture)

- CPU ↔ Bus : bus de contrôle (écriture)
- HDD ↔ RAM : bus de données via DMA (accès direct mémoire)
- Tous partagent le bus système (adresses + données + contrôle)

Schéma récapitulatif de l'Architecture simplifiée

```

graph TD
    subgraph "Unité Centrale (CPU)"
        UC[Unité de Contrôle] --> ALU[ALU]
        ALU --> REG[Registres]
    end

    subgraph "Mémoire"
        RAM[RAM] --> CACHE[Cache]
        CACHE --> REG
    end

    subgraph "Entrées / Sorties"
        ENTREE[Entrées / Sorties]
        SORTIE[Sorties]
        STOCKAGE[Stockage de masse]
    end

    UC --> ENTREE
    UC --> SORTIE
    RAM --> STOCKAGE

    style UC fill:#4a90d9,color:#fff
    style ALU fill:#e67e22,color:#000
    style REG fill:#9b59b6,color:#fff
    style RAM fill:#2ecc71,color:#fff
    style CACHE fill:#2ecc71,color:#fff

```

Exercice 2 – Processus et threads (18 min)

Scénario enseignant : Vos étudiants confondent souvent processus et thread. Utilisez l'exemple concret du navigateur web pour ancrer la différence. Proposez des analogies (usine/ouvrier, cuisine/chef).

Question de base (6 min)

Nonc : Distinguer processus et thread. Donner un exemple concret avec un navigateur web.

Corrigé détaillé :

Caractéristique	Processus	Thread
Espace mémoire	Indépendant (propre espace d'adressage)	Partagé avec le processus parent
Communication	Complexe (IPC : pipes, sockets, signaux)	Directe (variables globales)
Création	Lourde (copie du contexte)	Légère (simple fil d'exécution)
Isolation	Forte (un crash n'affecte pas les autres)	Faible (un bug peut tout corrompre)
Commutation	Coûteuse (changement de contexte)	Rapide (même espace d'adressage)

Analogies pédagogiques :

- **Processus = usine** (chaque usine a ses propres ressources, Ã©quipements, stocks)
- **Thread = ouvrier** (les ouvriers d'une mÃªme usine partagent les mÃªmes outils et locaux)

Exemple navigateur web :

- 1 **processus** pour l'ensemble du navigateur
- Plusieurs **threads** : UI (interface), rendu (affichage des pages), rÃ©seau (tÃ©lÃ©chargement), extensions (plugins)

â€¦ Question intermÃ©diaire (6 min)

Ã‰noncÃ© : Cochez la bonne colonne pour chaque situation.

Situation	Processus	Thread
Deux onglets Chrome isolÃ©s (site A et site B)	?	?
VÃ©rification orthographique dans un traitement de texte	?	?
Un serveur web qui traite 100 requÃªtes simultanÃ©es	?	?
Un programme malveillant qui fait planter tout le systÃ¨me	?	?

CorrigÃ© :

Situation	Processus	Thread
Deux onglets Chrome isolÃ©s	â€œ (isolation de sÃ©curitÃ©)	
VÃ©rification orthographique		â€œ (lÃ©ger, partage le document)
Serveur web 100 requÃªtes		â€œ (pool de threads pour performance)
Programme malveillant plante tout	â€œ (si un processus accÃ©de au noyau)	

â€¦â€¦ Question avancÃ©e (6 min)

Ã‰noncÃ© : Sous Linux, la commande `ps aux` liste les processus. InterprÃ©ter les lignes suivantes :

```
USER PID %CPU %MEM VSZ RSS COMMAND
madani 1234 12.5 2.3 250000 45000 firefox
madani 5678 0.1 0.1 12000 2500 bash
root 9012 0.0 0.5 80000 10000 systemd-journald
```

1. Quel processus consomme le plus de CPU ?
2. Lequel utilise le plus de mÃ©moire RAM ?
3. Pourquoi `bash` a-t-il un PID plus grand que `firefox` alors qu'il a Ã©tÃ© lancÃ© avant ?

CorrigÃ© :

1. `firefox` (PID 1234, 12,5 % CPU) â€™ c'est le navigateur avec rendu et scripts.
2. `firefox` (RSS 45 000 Ko = 45 Mo) â€™ un navigateur moderne est gourmand en mÃ©moire.
3. Les PID ne sont pas strictement croissants dans l'ordre de lancement : recyclage des PID, fork, etc. Le shell (`bash`) peut avoir Ã©tÃ© lancÃ© avant Firefox mais avoir un PID plus grand si Firefox a hÃ©ritÃ© d'un PID libÃ©rÃ©.

Diagramme comparatif

```

graph TB
  subgraph "Processus A"
    TH_A1[Thread 1] --> MEM_A[Mémoire A]
    TH_A2[Thread 2] --> MEM_A
    TH_A3[Thread 3] --> MEM_A
  end

  subgraph "Processus B"
    TH_B1[Thread 1] --> MEM_B[Mémoire B]
    TH_B2[Thread 2] --> MEM_B
  end

  CPU --> TH_A1
  CPU --> TH_A2
  CPU --> TH_A3
  CPU --> TH_B1
  CPU --> TH_B2

  style Processus A fill:#4a90d9,color:#fff
  style Processus B fill:#2ecc71,color:#fff
  style CPU fill:#e74c3c,color:#fff

```

Exercice 3 – Algorithme maximum (18 min)

Scénario enseignant : L'algorithmique est le cœur de la pensée informatique. Cet exercice simple (recherche du maximum) permet de valider la maîtrise des boucles, conditions, indices de tableau, et de la notion de complexité.

– Question de base (6 min)

Énoncé : Écrire en pseudo-code l'algorithme qui trouve le maximum dans un tableau de n entiers. Quelle est sa complexité temporelle ?

Corrigé détaillé :

```

ALGORITHME Maximum(T[0..n-1])
  ENTRÉE : T, tableau d'entiers de taille n (n ≥ 1)
  SORTIE : la valeur maximale du tableau

  max ← T[0] // Initialisation : on suppose que le premier est le max
  POUR i DE 1 À n-1 FAIRE // On parcourt les éléments restants
    SI T[i] > max ALORS // Si on trouve un élément plus grand
      max ← T[i] // On met à jour max
  FIN SI
  FIN POUR

  RETOURNER max
FIN ALGORITHME

```

Complexité : $O(n)$ – un seul parcours linéaire du tableau. C'est la complexité optimale car il faut examiner chaque élément au moins une fois.

Preuve de correction (Élémentaire) :

- *Initialisation* : $\text{max} = T[0]$ est bien le maximum du sous-tableau $T[0..0]$
- *Conservation* : à chaque itération i , max est le maximum de $T[0..i]$
- *Terminaison* : quand $i = n-1$, max est le maximum de tout le tableau

– Question intermédiaire (6 min)

À%noncÀ : Adapter l'algorithme pour retourner l'**indice** de la premiÃ`re occurrence du maximum (et non sa valeur).

CorrigÀ :

```
ALGORITHME IndiceMaximum(T[0..n-1])
max ← T[0]
indice ← 0
POUR i DE 1 À n-1 FAIRE
SI T[i] > max ALORS
max ← T[i]
indice ← i // On mémorise l'indice
FIN SI
FIN POUR
RETOURNER indice // Indice de la premiÃ`re occurrence
FIN ALGORITHME
```

Variante : Pour obtenir l'indice de la *derniÃ`re* occurrence, remplacer > par >=.

À...À... Question avancÀe (6 min)

À%noncÀ : À%crire un algorithme qui trouve **les deux plus grandes valeurs** d'un tableau en un **seul parcours** (complexitÀ $O(n)$).

Indice : maintenez deux variables max1 et max2.

CorrigÀ :

```
ALGORITHME DeuxMax(T[0..n-1])
max1 ← T[0] // La plus grande
max2 ← T[1] // La deuxiÃ`me plus grande

SI max2 > max1 ALORS // On s'assure que max1 ≤ max2
À%CHANGER max1 ET max2
FIN SI

POUR i DE 2 À n-1 FAIRE
SI T[i] > max1 ALORS
max2 ← max1
max1 ← T[i]
SINON SI T[i] > max2 ALORS
max2 ← T[i]
FIN SI
FIN POUR

RETOURNER (max1, max2)
FIN ALGORITHME
```

ComplexitÀ : toujours $O(n)$, mais avec un peu plus d'opÃ©rations par itÃ©ration.

Illustration À€ Parcours du tableau

```
graph LR
subgraph "Tableau T"
T0["T[0] = 3"] --> T1["T[1] = 7"]
T1 --> T2["T[2] = 2"]
T2 --> T3["T[3] = 9"]
T3 --> T4["T[4] = 1"]
end

subgraph "À%volution de max"
M0["max = 3"] --> M1["max = 7"]
M1 --> M2["max = 7"]
end
```

```

M2 --> M3["max = 9"]
M3 --> M4["max = 9"]
end

T0 --> M0
T1 --> M1
T2 --> M2
T3 --> M3
T4 --> M4

style T0 fill:#4a90d9,color:#fff
style T3 fill:#e74c3c,color:#fff
style M0 fill:#4a90d9,color:#fff
style M3 fill:#e74c3c,color:#fff

```

Exercice 4 – Réseaux (18 min)

Scénario enseignant : Les notions de base du réseau (IP, masque, passerelle, DNS) sont essentielles pour comprendre comment les machines communiquent. Utilisez une analogie postale (adresse = IP, code postal = masque, bureau de poste = passerelle, annuaire = DNS).

« Question de base (6 min)

Nonc : Différencier IP, masque de sous-réseau et passerelle par défaut. Expliquer le rôle du DNS.

Corrigé détaillé :

- **Adresse IP** (Internet Protocol) : identifiant unique d'une machine sur le réseau (ex: 192.168.1.10). Comme une adresse postale.
- **Masque de sous-réseau** : sépare la partie réseau de la partie hôte dans l'adresse IP (ex: 255.255.255.0 = /24). Comme le code postal.
- **Passerelle par défaut** (default gateway) : adresse IP du routeur qui permet de sortir du réseau local pour accéder à Internet (ex: 192.168.1.1). Comme le bureau de poste central.
- **DNS** (Domain Name System) : traduit un nom de domaine (google.com) en adresse IP (142.250.178.4). Comme l'annuaire téléphonique.

Exemple concret : Quand vous tapez `www.ens.dz` dans un navigateur :

1. Le navigateur demande au DNS : « quelle est l'IP de `www.ens.dz` ? »
2. Le DNS répond : `193.194.xxx.xxx`
3. Le navigateur envoie une requête HTTP à cette IP via la passerelle
4. Le serveur web répond avec la page HTML

... Question intermédiaire (6 min)

Nonc : Une machine a l'IP 192.168.1.35 avec un masque 255.255.255.0.

1. Quelle est l'adresse du réseau ?
2. Combien de machines peuvent être connectées sur ce réseau ?
3. La machine 192.168.1.200 est-elle sur le même réseau ?

4. Et la machine 192.168.2.10 ?

Corrigé :

1. **Adresse réseau** : 192.168.1.0 (partie réseau = 192.168.1, partie hôte = 0).

Calcul : IP & Masque = 192.168.1.35 & 255.255.255.0 = 192.168.1.0

2. **Nombre de machines** : $2^8 - 2 = 254$ machines (car 8 bits pour la partie hôte, on exclut l'adresse réseau .0 et le broadcast .255).

3. **Oui**, 192.168.1.200 a la même partie réseau (192.168.1). Elle est sur le même réseau local.

4. **Non**, 192.168.2.10 a une partie réseau différente (192.168.2). Elle nécessite un routage via la passerelle.

... Question avancée (6 min)

non : On considère le réseau 172.16.0.0/16. On veut créer 4 sous-réseaux de taille égale.

1. Quel nouveau masque faut-il utiliser ?

2. Quelles sont les adresses de chaque sous-réseau ?

3. Combien de machines par sous-réseau ?

Corrigé :

1. Masque /16 à /18 (on emprunte 2 bits pour le sous-réseage). Soit 255.255.192.0.

2. Les 4 sous-réseaux :

- 172.16.0.0/18 (hôtes : 172.16.0.1 à 172.16.63.254)
- 172.16.64.0/18 (hôtes : 172.16.64.1 à 172.16.127.254)
- 172.16.128.0/18 (hôtes : 172.16.128.1 à 172.16.191.254)
- 172.16.192.0/18 (hôtes : 172.16.192.1 à 172.16.255.254)

3. $2^8 - 2 = 16\,382$ machines par sous-réseau.

Illustration de l'change client-serveur

```
sequenceDiagram
    participant C as Client (Navigateur)
    participant DNS as Serveur DNS
    participant G as Passerelle (Routeur)
    participant S as Serveur Web

    C->>DNS: Quelle est l'IP de www.ens.dz ?
    DNS-->>C: Réponse : 193.194.x.x
    C->>G: Requête HTTP GET /index.html
    G->>S: Transmission via Internet
    S-->>G: Réponse HTTP 200 + page HTML
    G-->>C: Transmission de la page
    Note over C,S: Le DNS n'intervient qu'une fois (à solution initiale)
```

Exercice 5 de Python et sécurité (16 min)

Scénario enseignant : Vos étudiants débutent en Python. Proposez un script simple (moyenne de notes) pour vérifier les bases. Ajoutez une dimension sécurité pour sensibiliser dès la 1ère

sécurité.

Question de base (6 min)

Nonc : Écrire un script Python qui calcule la moyenne de 5 notes. Lister 3 bonnes pratiques en sécurité informatique.

Corrigé d'attente :

```
# Script : calcul_moyenne.py
# Auteur : Étudiant ENS
# Date : 2025

notes = [12, 14, 11, 15, 13] # Liste des 5 notes
moyenne = sum(notes) / len(notes) # Calcul de la moyenne
print(f"Moyenne : {moyenne:.2f}") # Affichage formaté à 2 décimales
```

Explication pas à pas :

1. `notes = [...]` : création d'une liste Python (tableau dynamique)
2. `sum(notes)` : fonction intégrée qui additionne tous les éléments
3. `len(notes)` : retourne la taille de la liste (ici 5)
4. `moyenne = ...` : affectation du résultat
5. `print(f"...")` : f-string pour formater l'affichage

Bonnes pratiques de sécurité :

#	Pratique	Pourquoi ?
1	Authentification multifacteur (MFA)	Ajoute une couche de sécurité au-delà du mot de passe
2	Mises à jour régulières	Corrigent les vulnérabilités connues (zero-day, failles)
3	Sauvegardes automatiques	Permettent de restaurer les données en cas d'attaque (ransomware)
4	Mots de passe uniques et complexes	Empêchent le credential stuffing (réutilisation sur plusieurs sites)
5	Principe du moindre privilège	Chaque utilisateur/programme n'a que les droits strictement nécessaires

Question intermédiaire (5 min)

Nonc : Modifier le script Python pour qu'il :

1. Demande les notes à l'utilisateur via `input()` (5 notes)
2. Vérifie que chaque note est comprise entre 0 et 20
3. Affiche un message d'erreur si une note est invalide

Corrigé :

```
notes = []
for i in range(5):
    try:
        note = float(input(f"Entrez la note {i+1} (0-20) : "))
        if 0 <= note <= 20:
            notes.append(note)
        else:
            print("Erreur : la note doit être entre 0 et 20.")
            break
    except ValueError:
```

```

print("Erreur : veuillez entrer un nombre valide.")
break

if len(notes) == 5:
    moyenne = sum(notes) / len(notes)
    print(f"Moyenne : {moyenne:.2f}")
else:
    print("Calcul impossible : notes incomplètes.")

```

Concepts Python utilisés :

- `input()` : saisie utilisateur (retourne une chaîne)
- `float()` : conversion en nombre décimal
- `try/except` : gestion d'exceptions (si l'utilisateur entre du texte)
- `break` : sortie anticipée de la boucle

... Question avancée (5 min)

nonc : En une ligne de Python (compréhension de liste), calculer la moyenne d'un nombre variable de notes passées en argument via `sys.argv`.

Corrigé :

```

import sys
notes = [float(x) for x in sys.argv[1:]]
print(f"Moyenne : {sum(notes)/len(notes):.2f}" if notes else "Aucune note fournie.")

```

Exécution : `python moyenne.py 12 14 11 15 13`

Résultat : Moyenne : 13.00

Diagramme "Sécurité en couches"

```

graph BT
    subgraph "Couches de sécurité"
        MFA["MFA<br/>Authentification"] --> MDP["Mots de passe<br/>forts et uniques"]
        MDP --> MAJ["Mises à jour<br/>régulières"]
        MAJ --> SAUV["Sauvegardes<br/>automatiques"]
        SAUV --> PRIV["Principe du<br/>moindre privilège"]
    end

    subgraph "Menaces"
        RAN["Ransomware"] -.-> SAUV
        CRED["Credential stuffing"] -.-> MDP
        VULN["Exploitation de failles"] -.-> MAJ
        PHISH["Phishing"] -.-> MFA
    end

    style MFA fill:#4a90d9,color:#fff
    style MDP fill:#2ecc71,color:#fff
    style MAJ fill:#e67e22,color:#000
    style SAUV fill:#9b59b6,color:#fff
    style PRIV fill:#e74c3c,color:#fff

```

QCM d'auto-évaluation

1. Le cycle d'exécution du CPU commence par :

a) Execute b) Fetch c) Decode

2. Un thread partage la mémoire de :

a) Un autre thread b) Son processus parent c) Tous les processus

3. Quel algorithme de tri a une complexité $O(n^2)$?

a) Tri fusion b) Tri à bulles c) Recherche dichotomique

4. Le DNS traduit :

a) Une IP en MAC b) Un nom en IP c) Une URL en HTML

5. Quel mot-clé définit une fonction en Python ?

a) function b) def c) define

6. Quel est le rôle du masque de sous-réseau ?

a) Chiffrer les données b) Séparer réseau et hôte c) Accrocher la connexion

7. Un processus à l'état « Attenué » signifie que :

a) Il attend une ressource b) Il est exécuté par le CPU c) Il vient d'être créé

8. En Python, `len([1, 2, 3])` retourne :

a) 2 b) 3 c) 4

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Architecture machine	20 %	Ne distingue pas les composants	Composants partiellement identifiés	Composants identifiés
Processus et OS	20 %	Ne différencie pas processus/programme	Concepts partiellement maîtrisés	Concepts maîtrisés
Algorithmique	20 %	Aucun algorithme correct	Algorithmes partiellement corrects	Algorithmes corrects
Réseaux	20 %	Ne comprend pas le modèle client-serveur	Concepts réseau partiellement compris	Concepts réseau compris
Python	20 %	Aucun script fonctionnel	Script partiellement fonctionnel	Script fonctionnel

Note pédagogique ENS

Transposition didactique

Ce TD de synthèse montre l'interconnexion des thèmes du module. En classe de Seconde (SNT), on peut proposer un TD similaire mais avec des exercices simplifiés : conversion binaire sur de petits nombres, reconnaissance des périphériques, utilisation d'un navigateur pour illustrer le client-serveur. L'objectif est de donner une vision d'ensemble sans entrer dans les détails techniques.

Conseil pour l'enseignant formateur

Utiliser ce TD comme évaluation diagnostique en début de module. Demander aux étudiants de répondre sans préparation, puis corriger collectivement pour mesurer le niveau initial. Les résultats permettront d'adapter la progression des séances suivantes. Les pictogrammes « / » permettent une différenciation pédagogique : les étudiants rapides peuvent avancer sur les « / » pendant que les autres terminent les « ».

Prolongement

Réaliser une carte mentale collective (sur papier ou avec un outil comme XMind) qui relie tous les thèmes abordés. Chaque étudiant ajoute une notion ou un exemple vu dans le TD. Cette carte servira de fil conducteur tout au long du module.

Pour aller plus loin

- **Architecture** : Simuler le cycle d'exécution avec un jeu de rôle (un étudiant = chaque composant)
- **Processus** : Utiliser le Gestionnaire de tâches Windows (`Ctrl+Maj+Esc`) pour observer les processus en direct
- **Algorithme** : Implémenter le maximum en Python et tester sur des tableaux aléatoires de taille croissante (mesurer le temps d'exécution avec `time`)
- **Réseaux** : Utiliser `ping`, `tracert`, `nslookup` en ligne de commande pour observer le DNS et le routage
- **Python** : Proposer un mini-projet « calculatrice de moyenne avec notes pondérées » qui investit les listes, boucles et conditions

Templates : `Template/PG-fr.docx` | `Template/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_FR_02.pdf`

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 02 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- Convertir un nombre dÃ©cimal en binaire et inversement
- Convertir entre les bases binaire, dÃ©cimale et hexadÃ©cimale
- DÃ©coder des caractÃ¨res ASCII Ã partir d'une sÃ©quence binaire
- Estimer la taille de diffÃ©rents types de fichiers (texte, image, audio)
- Utiliser les prÃ©fixes d'unitÃ©s (Ko, Mo, Go) et calculer des rapports de compression

Contexte pÃ©dagogique â€” ScÃ©nario enseignant

Vous Ãªtes enseignant et devez faire dÃ©couvrir Ã vos Ã©tudiants **le monde des conversions numÃ©riques et du codage de l'information**. Ce TD alterne exercices de conversion (binaire, dÃ©cimal, hexadÃ©cimal) et applications concrÃ¨tes (codage ASCII, taille des fichiers). PrÃ©voyez des **cartes des puissances de 2** (1, 2, 4, 8, 16, 32, 64, 128, 256) pour que les Ã©tudiants manipulent physiquement les conversions.

Symbole	Niveau	Attendu
â€œ	DÃ©butant	Questions de base â€” tous les Ã©tudiants doivent rÃ©pondre
âˆ…	IntermÃ©diaire	Approfondissement â€” Ã traiter si le temps le permet
âˆ…âˆ…	AvancÃ©	DÃ©fi â€” pour les plus rapides, peut faire l'objet d'un bonus

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” Conversions binaires	15 min	20 min
Exercice 2 â€” Conversions dÃ©cimales	15 min	35 min
Exercice 3 â€” HexadÃ©cimal	15 min	50 min
Exercice 4 â€” Codage des caractÃ¨res	15 min	65 min
Exercice 5 â€” Taille des fichiers	20 min	85 min
Correction	5 min	90 min
Total	90 min	

Conversion d'un décimal en binaire

```

flowchart TD
    A[Nombre décimal : 42] --> B{Diviser par 2}
    B --> C[Reste = 0]
    B --> D[Reste = 1]
    C --> E[Noter le reste]
    D --> E
    E --> F{Quotient = 0 ?}
    F -->|Non| B
    F -->|Oui| G[Lire les restes<br/>à l'envers]
    G --> H[Résultat binaire : 101010]

    style A fill:#4a90d9,color:#fff
    style H fill:#2ecc71,color:#fff
    style B fill:#e67e22,color:#000
    style G fill:#9b59b6,color:#fff

```

Tableau des puissances de 2 (référence)

2^n	Valeur	Binaire
2^0	1	1
2^1	2	10
2^2	4	100
2^3	8	1000
2^4	16	1 0000
2^5	32	10 0000
2^6	64	100 0000
2^7	128	1 000 0000
2^8	256	1 0000 0000
2^9	512	10 0000 0000
2^{10}	1024	100 0000 0000

Hierarchie des unités de stockage

```

graph LR
    subgraph "Unités de stockage"
        bit --> Octet[1 octet = 8 bits]
        Octet --> Ko[1 Ko = 1024 octets]
        Ko --> Mo[1 Mo = 1024 Ko]
        Mo --> Go[1 Go = 1024 Mo]
        Go --> To[1 To = 1024 Go]
    end

    subgraph "Exemples"
        Texte["Texte 5000 car.<br/>~ 5 Ko"]
        ImageNB["Image NB 200×150<br/>~ 3,7 Ko"]
        ImageC["Image 24 bits 200×150<br/>~ 88 Ko"]
        Photo["Photo 4000×3000<br/>~ 34,3 Mo"]
    end

```

```

Ko --> Texte
Ko --> ImageNB
Mo --> ImageC
Mo --> Photo

style bit fill:#4a90d9,color:#fff
style To fill:#e74c3c,color:#fff
style Texte fill:#2ecc71,color:#fff
style ImageNB fill:#2ecc71,color:#fff
style ImageC fill:#2ecc71,color:#fff
style Photo fill:#2ecc71,color:#fff

```

Exercice 1 – Conversions binaires (15 min)

Scénario enseignant : Commencez par la méthode des divisions successives. Distribuez la table des puissances de 2 (ci-dessus). Pour chaque nombre, demandez aux étudiants de décomposer en sommes de puissances de 2, puis de passer à la méthode posée.

Question de base (6 min)

Nononcé : Convertir les nombres suivants en binaire par la méthode des **divisions successives par 2**.

- 13 (décimal) à binaire
- 25 (décimal) à binaire
- 42 (décimal) à binaire
- 100 (décimal) à binaire

Corrigé détaillé :

1. 13, à :

```

13 ÷ 2 = 6 reste 1 (poids faible)
6 ÷ 2 = 3 reste 0 |
3 ÷ 2 = 1 reste 1 |
1 ÷ 2 = 0 reste 1 | (poids fort)

```

Lecture des restes de bas en haut : **1101**,

Vérification : $8 + 4 + 1 = 13$ ✓

2. 25, à :

```

25 ÷ 2 = 12 reste 1
12 ÷ 2 = 6 reste 0
6 ÷ 2 = 3 reste 0
3 ÷ 2 = 1 reste 1
1 ÷ 2 = 0 reste 1

```

à **11001**, ($16 + 8 + 1 = 25$ ✓)

3. 42, à :

```

42 ÷ 2 = 21 reste 0
21 ÷ 2 = 10 reste 1
10 ÷ 2 = 5 reste 0
5 ÷ 2 = 2 reste 1
2 ÷ 2 = 1 reste 0
1 ÷ 2 = 0 reste 1

```


â†’ 101010â,, (32 + 8 + 2 = 42 âœ“)

4. 100â,â,€ :

100 ÷ 2 = 50 reste 0
 50 ÷ 2 = 25 reste 0
 25 ÷ 2 = 12 reste 1
 12 ÷ 2 = 6 reste 0
 6 ÷ 2 = 3 reste 0
 3 ÷ 2 = 1 reste 1
 1 ÷ 2 = 0 reste 1

â†’ 1100100â,, (64 + 32 + 4 = 100 âœ“)

â… Question intermÃ©diaire (5 min)

Ã‰noncÃ© : Convertir en binaire **sans poser les divisions**, en utilisant uniquement les puissances de 2.

- 31
- 64
- 255

CorrigÃ© :

- 31 = 16 + 8 + 4 + 2 + 1 = 11111â,, (5 bits ã 1 = 2âµ - 1)
- 64 = 1000000â,, (1 suivi de 6 zÃ©ros = 2â¶)
- 255 = 256 - 1 = 2â¶ - 1 = 11111111â,, (8 bits ã 1)

Astuce : 2â¶ - 1 en binaire = n bits ã 1. Exemple : 2âµ - 1 = 31 = 11111â,,.

â…â… Question avancÃ©e (4 min)

Ã‰noncÃ© : Convertir 255.255.255.0 (masque de sous-rÃ©seau en notation dÃ©cimale pointÃ©e) en binaire.

CorrigÃ© :

255 = 11111111â,,
 255 = 11111111â,,
 255 = 11111111â,,
 0 = 00000000â,,

Soit : 11111111.11111111.11111111.00000000â,, (masque /24)

Tableau Ã© complÃ©ter (auto-entraÃ®nement)

DÃ©cimal	DÃ©composition en puissances de 2	Binaire
7	4+2+1	
12		
37		
200		

CorrigÃ© : 7 = 111â,, , 12 = 1100â,, , 37 = 100101â,, , 200 = 11001000â,,.

Exercice 2 – Conversions décimales (15 min)

Scénario enseignant : L'opération inverse (binaire à décimal) est plus intuitive : il suffit d'additionner les puissances de 2. Montrez la correspondance entre la position du bit et la puissance de 2.

Question de base (6 min)

Nonc : Convertir les nombres binaires suivants en décimal (additionner les puissances de 2).

1. 1010₂ , ,
2. 1111₂ , ,
3. 100000₂ , ,
4. 110011₂ , ,

Corrigé détaillé :

1. $1010_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 8 + 2 = 10$
2. $1111_2 = 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 4 + 2 + 1 = 15$
3. $100000_2 = 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 32$
4. $110011_2 = 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 32 + 16 + 2 + 1 = 51$

Méthode du doublement (alternative) :

Partir de 0, pour chaque bit de gauche à droite : doubler le résultat et ajouter le bit.

- 1010₂ : 0 à 0×2+1=1 à 1×2+0=2 à 2×2+1=5 à 5×2+0=10

Question intermédiaire (5 min)

Nonc : Quel est le plus grand nombre décimal qu'on peut coder avec :

1. 4 bits ?
2. 8 bits ?
3. 16 bits ?

Corrigé :

1. 4 bits : $1111_2 = 2^4 - 1 = 15$
2. 8 bits : $1111\ 1111_2 = 2^8 - 1 = 255$
3. 16 bits : $2^{16} - 1 = 65535$

Règle générale : Avec n bits, on peut coder les nombres de 0 à $2^n - 1$, soit 2^n valeurs différentes.

Question avancée (4 min)

Nonc : Le nombre binaire 1010 1100 0011 1110₂ est trop long. Proposez une méthode pour le convertir plus rapidement en passant par une base intermédiaire. Combien vaut-il en décimal ?

Indice : Groupez par 4 bits de la droite vers la gauche.

Corrigé :

1010 1100 0011 1110
 A C 3 E , (hexadécimal)

$$AC3E_{16} = 10 \times 4096 + 12 \times 256 + 3 \times 16 + 14 = 40960 + 3072 + 48 + 14 = 44094_{10}$$

Tableau d'entrainement (binaire à décimal)

Binaire	Poids forts à faibles	Décimal
0011	0+0+2+1	
1011		
1110 0101		
0010 0000 0001		

Corrigé : 0011 = 3, 1011 = 11, 1110 0101 = 229, 0010 0000 0001 = 513.

Exercice 3 – Hexadécimal (15 min)

Scénario enseignant : L'hexadécimal (base 16) est omniprésent en informatique (couleurs HTML, adresses MAC, mémoire). La clé est le passage par le binaire : 1 chiffre hexa = 4 bits. Distribuez la table de correspondance.

Table de correspondance binaire à hexadécimal

Décimal	Binaire	Hexa
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Question de base (6 min)

Nonc : Convertir en hexadécimal.

- 255 (décimal) → hexa
- 16 (décimal) → hexa
- 1101 1010₂ , , → hexa
- A3₁₆, 10₁₆, 3 → décimal

Corrigé détaillé :

1. 255₁₀ → hexa :

$255 \div 16 = 15$ reste 15 → FF₁₆, 10₁₆, 3 (15 = F en hexa).

Soit FF₁₆, 10₁₆, 3 = $15 \times 16 + 15 = 255$ ✓

2. 16₁₀ → hexa :

$16 \div 16 = 1$ reste 0 → 10₁₆, 10₁₆, 3 (attention : 10 en hexa = 16, pas 10 !)

3. 1101 1010₂ , , → hexa :

Grouper par 4 bits : 1101₂ , , = D₁₆, 10₁₆, 3, 1010₂ , , = A₁₆, 10₁₆, 3.

Soit DA₁₆, 10₁₆, 3

4. A3₁₆, 10₁₆, 3 → décimal :

A = 10, donc A3₁₆, 10₁₆, 3 = $10 \times 16 + 3 = 160 + 3 = 163$

Question intermédiaire (5 min)

Nonc : Les couleurs HTML sont codées en hexadécimal sur 3 octets (R, V, B). Par exemple, #FF0000 = rouge pur. Décoder :

- #00FF00
- #0000FF
- #FFFFFF
- #000000

Corrigé :

- #00FF00 → R=0, V=255, B=0 → **Vert pur**
- #0000FF → R=0, V=0, B=255 → **Bleu pur**
- #FFFFFF → R=255, V=255, B=255 → **Blanc**
- #000000 → R=0, V=0, B=0 → **Noir**

Question avancée (4 min)

Nonc : Adresse MAC : 3A:1B:7E:00:FF:4D. Convertir chaque octet en binaire.

Corrigé :

3A₁₆, 10₁₆, 3 = 0011 1010₂ , ,
 1B₁₆, 10₁₆, 3 = 0001 1011₂ , ,
 7E₁₆, 10₁₆, 3 = 0111 1110₂ , ,
 00₁₆, 10₁₆, 3 = 0000 0000₂ , ,
 FF₁₆, 10₁₆, 3 = 1111 1111₂ , ,
 4D₁₆, 10₁₆, 3 = 0100 1101₂ , ,

4Dâ,□â,† = 0100 1101â,,

Soit : 00111010 00011011 01111110 00000000 11111111 01001101â,,

Exercice 4 – Codage des caractères (15 min)

Scénario enseignant : Les étudiants utilisent quotidiennement du texte sans savoir comment les caractères sont stockés. Montrez la table ASCII (7 bits) et son extension sur 8 bits. Cet exercice concret d'écoder un message caché.

Question de base (6 min)

Nonc : En ASCII (7 bits), A = 65, a = 97, espace = 32. Décoder la séquence binaire suivante :

01000001 00100000 01000010 01001111 01001110

Indice : Chaque groupe de 8 bits = 1 caractère.

Corrigé d'écoder :

Binaire (8 bits)	Décimal	Caractère ASCII
01000001â,	65	A
00100000â,	32	 (espace)
01000010â,	66	B
01001111â,	79	O
01001110â,	78	N

Résultat : A BON

Méthode : Pour chaque octet, lire la valeur binaire et chercher dans la table ASCII (ou calculer : A=65 donc B=66, O=79, N=78).

Question intermédiaire (5 min)

Nonc : Coder votre prénom en binaire ASCII (8 bits par caractère).

Exemple : MADANI = ?

Corrigé pour MADANI :

M = 77â,□â,€ = 01001101â,,
 A = 65â,□â,€ = 01000001â,,
 D = 68â,□â,€ = 01000100â,,
 A = 65â,□â,€ = 01000001â,,
 N = 78â,□â,€ = 01001110â,,
 I = 73â,□â,€ = 01001001â,,

Soit : 01001101 01000001 01000100 01000001 01001110 01001001

Question avancée (4 min)

Nonc : Décoder le message suivant (il utilise ASCII 7 bits, mais le bit de poids fort est utilisé comme bit de parité paire – ignorable) :

11000010 11100101 11100100 11100101 11101110 11100101

Attention : Ne garder que les 7 bits de poids faible.

Corrigé :

```
11000010 à ' 1000010â,, = 66 = 'B'
11100101 à ' 1100101â,, = 101 = 'e'
11100100 à ' 1100100â,, = 100 = 'd'
11100101 à ' 1100101â,, = 101 = 'e'
11101110 à ' 1101110â,, = 110 = 'n'
11100101 à ' 1100101â,, = 101 = 'e'
```

Résultat : Bedene

Exercice 5 – Taille des fichiers (20 min)

Scénario enseignant : Exercice très concret : les étudiants pourront vérifier les résultats en ouvrant les propriétés de leurs propres fichiers. Expliquez la différence entre **taille non compressée** (calcul théorique) et **taille réelle** (après compression : JPEG, MP3, etc.).

Question de base (8 min)

Nonc : Calculer la taille estimée des fichiers suivants.

1. Un texte de 5000 caractères ASCII (sans mise en forme).
2. Une image 200—150 pixels en noir et blanc (1 bit/pixel).
3. Une image 200—150 pixels en 24 bits (TrueColor).
4. Une photo 4000—3000 pixels en 24 bits compressée JPEG (ratio 10:1).

Corrigé de taille :

1. **Texte :** 5000 — 1 octet = **5000 octets**

Soit 5000 — 1024 ≈ **4,9 Ko**

2. **Image NB 200—150 :**

- Nombre de pixels = 200 — 150 = 30 000 pixels
- 1 bit/pixel à ' 30 000 bits = 30 000 — 8 = **3750 octets**
- Soit 3750 — 1024 ≈ **3,7 Ko**

3. **Image couleur 200—150 (24 bits) :**

- 30 000 pixels — 24 bits = 720 000 bits
- 720 000 — 8 = **90 000 octets**
- Soit 90 000 — 1024 ≈ **87,9 Ko**

Remarque : une icône typique (32—32) en 24 bits = 32—32—3 = 3072 octets ≈ 3 Ko.

4. **Photo 4000—3000 (24 bits) compressée JPEG :**

- 4000 — 3000 = 12 000 000 pixels
- 12 000 000 — 3 = 36 000 000 octets ≈ **34,3 Mo**
- Compression JPEG (10:1) à ' 36 000 000 — 10 = **3 600 000 octets** ≈ **3,4 Mo**

Formules essentielles à retenir :

- Taille (octets) = Largeur — Hauteur (bits/pixel) — 8

- Taille compressée = Taille non compressée $\cdot$ ratio de compression

Question intermédiaire (6 min)

non : Une chanson au format CD audio (44,1 kHz, 16 bits, stéréo) dure 3 minutes. Calculer :

1. Le débit audio en bits par seconde (bitrate).
2. La taille totale du fichier non compressé.
3. La taille compressée en MP3 (bitrate 128 kbps $\approx$ kilo bits par seconde).

Corrigé :

1. Débit audio :

- $44\,100$ échantillons/s $\times$ 16 bits/échantillon $\times$ 2 canaux = **1 411 200 bits/s**
- Soit $\approx$ **1,41 Mbps** (Mega bits par seconde)

2. Taille non compressée :

- $1\,411\,200$ bits/s $\times$ 180 s = $254\,016\,000$ bits
- $\times 8 = 31\,752\,000$ octets
- $\times 1024 \approx$ **31 008 Ko $\approx$ 30,3 Mo**

3. Taille compressée MP3 (128 kbps) :

- $128\,000$ bits/s $\times$ 180 s = $23\,040\,000$ bits
- $\times 8 = 2\,880\,000$ octets
- $\times 1024 \approx$ **2812 Ko $\approx$ 2,75 Mo**

Ratio de compression : $30,3 / 2,75 \approx$ **11:1** $\approx$ impressionnant !

Question avancée (6 min)

non : Un vidéoprojecteur affiche en 1920×1080 pixels, 24 bits, 60 images par seconde (ips). Pendant une présentation de 1 heure :

1. Quel est le débit vidéo non compressé ?
2. Quelle serait la taille totale non compressée ?
3. Avec un codec H.264 (ratio 100:1), quelle est la taille finale ?
4. Cela tient-il sur une clé USB de 32 Go ?

Corrigé :

1. Débit vidéo :

- $1920 \times 1080 = 2\,073\,600$ pixels/image
- $2\,073\,600 \times 24$ bits = $49\,766\,400$ bits/image
- $\times 60$ ips = $2\,985\,984\,000$ bits/s $\approx$ **2,99 Gbps**

2. Taille non compressée pour 1 h :

- $2\,985\,984\,000 \times 3600 = 10\,749\,542\,400\,000$ bits
- $\times 8 = 1\,343\,692\,800\,000$ octets $\approx$ **1,34 To**

3. Avec H.264 (100:1) :

- 1,34 To $\approx$ 100 $\times$ 13,4 Go

4. Oui, 13,4 Go < 32 Go, cela tient sur la clé USB.

Remarque : sans compression, il faudrait plusieurs disques durs !

Synthèse visuelle – Comparaison des tailles

```
graph BT
    subgraph "Ordres de grandeur"
        CHANSON["Chanson 3 min MP3<br/>~ 3 Mo"] --> PHOTO_JPEG["Photo 12 MP JPEG<br/>~ 4 Mo"]
        PHOTO_JPEG --> DOC["Document Word<br/>~ 50 Ko"]
        DOC --> MAIL["Email texte<br/>~ 5 Ko"]
        MAIL --> TEXTE["1 page ASCII<br/>~ 3 Ko"]
    end

    subgraph "Supports de stockage"
        CLE["Clé USB 32 Go"] --> DVD["DVD 4,7 Go"]
        DVD --> CD["CD 700 Mo"]
        CD --> DISQUETTE["Disquette 1,44 Mo"]
    end

    style CLE fill:#4a90d9,color:#fff
    style DVD fill:#2ecc71,color:#fff
    style CD fill:#e67e22,color:#000
    style DISQUETTE fill:#e74c3c,color:#fff
```

QCM d'auto-évaluation

- Quelle est la valeur décimale de 1010_2 , ?
a) 8 b) 10 c) 12
- Combien de bits faut-il pour coder le nombre 256 en binaire ?
a) 8 b) 9 c) 10
- Le nombre hexadécimal FF_{16} , $\square_{16}$, + vaut :
a) 240 b) 255 c) 256
- En ASCII, le caractère A est codé par :
a) 65 b) 97 c) 32
- Un Go (gigaoctet) vaut :
a) 1000 Mo b) 1024 Mo c) 1024 Ko
- $1 \text{ kibioctet (Kio)}$ vaut :
a) 1000 octets b) 1024 octets c) 1000 bits
- Une image 100×100 pixels en 24 bits pèse environ :
a) 10 Ko b) 30 Ko c) 100 Ko
- Le nombre hexadécimal $1F_{16}$, $\square_{16}$, + vaut en décimal :
a) 16 b) 31 c) 21

Grille d'Évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Conversion binaire à décimal	20 %	Conversions incorrectes	Conversions partiellement correctes	Conversions
Conversion hexadécimale	20 %	Conversions incorrectes	Conversions partiellement correctes	Conversions
Codage ASCII/UTF-8	20 %	Ne comprend pas le principe	Compréhension partielle	Compréhension
Calcul de taille fichier	20 %	Calculs incorrects	Calculs partiellement corrects	Calculs corrects
Image bitmap	20 %	Ne comprend pas le codage pixel	Compréhension partielle	Compréhension

Note pédagogique ENS

Transposition didactique

Les conversions binaires et hexadécimales sont abordables dès la classe de Seconde (SNT) en utilisant la méthode des divisions successives ou des puissances de 2. On peut introduire le binaire avec des jeux de cartes (cartes valant 1, 2, 4, 8, 16...) et faire deviner un nombre choisi par un élève. Pour les tailles de fichiers, utiliser des exemples concrets (photo du téléphone, document Word) que les étudiants manipulent quotidiennement.

Conseil pour l'enseignant formateur

Prévoir des exercices progressifs : commencer par des petits nombres (0-15) faciles à vérifier mentalement, puis augmenter graduellement. Pour l'hexadécimal, insister sur le passage par le binaire (grouper par 4 bits). L'exercice 5 (tailles) est très concret : demander aux étudiants d'ouvrir les propriétés de leurs propres fichiers pour vérifier les calculs. Les exercices sur l'audio et la vidéo (à... à...) peuvent être traités en fin de séance ou reportés en séance suivante si le temps manque.

Prolongement

Proposer un mini-projet : créer un petit script Python qui convertit automatiquement un nombre décimal vers les bases 2, 10 et 16, et affiche les étapes de calcul. Cela réinvestit les notions de boucles et conditions vues en algorithmique tout en consolidant les conversions de bases.

Pour aller plus loin

- **Binaire** : Jouer au « binaire humain » où 5 étudiants tiennent des pancartes 16, 8, 4, 2, 1 ; l'enseignant annonce un nombre, les étudiants lèvent ou baissent leur pancarte.
- **Hexadécimal** : Décoder l'adresse MAC de son propre ordinateur (commande `ipconfig /all` sur Windows).
- **Taille des fichiers** : Télécharger une image en différentes résolutions et comparer les tailles réelles avec les calculs théoriques.
- **Conversion Python** : Implémenter un convertisseur universel (base 2, 10, 16) avec interface texte.

Templates : [Template/PG-fr.docx](#) | [Template/Presentation_BELACEL_V3.pptx](#)

PDF : [TD_Informatique_ENS_1ère_année_FR_03.pdf](#)

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 03 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- Identifier et dÃ©crire les 5 unitÃ©s du modÃ¨le de von Neumann
- Ordonner les Ã©tapes du cycle d'exÃ©cution d'une instruction (fetch-decode-execute-writeback)
- Classer les diffÃ©rents types de mÃ©moire par vitesse et taille
- ExÃ©cuter des commandes de base en ligne de commande (PowerShell)
- Expliquer le rÃ´le de chaque composant dans le fonctionnement global d'un ordinateur

Contexte pÃ©dagogique â€” ScÃ©nario enseignant

Vous Ãªtes enseignant et devez expliquer **comment fonctionne un ordinateur de l'intÃ©rieur**. Ce TD dÃ©taille l'architecture de von Neumann, le cycle d'exÃ©cution, la hiÃ©rarchie mÃ©moire et les commandes de base en ligne de commande. Utilisez des **analogies concrÃ¨tes** (recette de cuisine, bureau de travail) et privilÃ©giez la **manipulation** (ligne de commande en direct si possible sur machine virtuelle).

Symbole	Niveau	Attendu
â€œ	DÃ©butant	Questions de base â€” tous les Ã©tudiants doivent rÃ©pondre
âˆ…	IntermÃ©diaire	Approfondissement â€” Ã traiter si le temps le permet
âˆ…âˆ…	AvancÃ©	DÃ©fi â€” pour les plus rapides, peut faire l'objet d'un bonus

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” von Neumann	15 min	20 min
Exercice 2 â€” Cycle d'exÃ©cution	15 min	35 min
Exercice 3 â€” HiÃ©rarchie mÃ©moire	20 min	55 min
Exercice 4 â€” Ligne de commande	15 min	70 min
Exercice 5 â€” QCM Architecture	15 min	85 min
Correction	5 min	90 min
Total	90 min	

Modèle de von Neumann – Diagramme détaillé

```

block-beta
columns 3
UC["Unité de Contrôle<br/>(Contrôle)"] space ALU["ALU<br/>(Traitement)"]
block:Memoire["Mémoire centrale<br/>(RAM)"]
columns 1
Instructions["Instructions"]
Donnees["Données"]
end
Entree["Entrée<br/>(Clavier)"] space Sortie["Sortie<br/>(Écran)"]

Bus["Bus système"] across

UC --> Bus
ALU --> Bus
Memoire --> Bus
Entree --> Bus
Sortie --> Bus

style UC fill:#4a90d9,color:#fff
style ALU fill:#e67e22,color:#000
style Memoire fill:#2ecc71,color:#fff
style Entree fill:#9b59b6,color:#fff
style Sortie fill:#9b59b6,color:#fff
style Bus fill:#e74c3c,color:#fff

```

Cycle d'exécution et hiérarchie mémoire

```

flowchart LR
subgraph "Cycle d'exécution"
direction LR
A[Fetch<br/>Chargement] --> B[Decode<br/>Décodage]
B --> C[Execute<br/>Exécution]
C --> D[Write-back<br/>Écriture]
end

subgraph "Hiérarchie mémoire"
direction TB
R[Registres<br/>512 octets < 1 ns] --> L1[Cache L1<br/>64 Ko octets ~1 ns]
L1 --> L2[Cache L2<br/>512 Ko octets ~4 ns]
L2 --> RAM[RAM<br/>16 Go octets ~50 ns]
RAM --> SSD[SSD<br/>1 To octets ~100 µs]
SSD --> HDD[HDD<br/>2 To octets ~10 ms]
end

D --> R

style A fill:#4a90d9,color:#fff
style B fill:#2ecc71,color:#fff
style C fill:#e67e22,color:#000
style D fill:#9b59b6,color:#fff
style R fill:#e74c3c,color:#fff
style HDD fill:#e74c3c,color:#fff

```

Schéma fonctionnel – Les bus de communication

```

graph TB
subgraph "Processeur"
UC[Unité de Contrôle]

```

```

ALU[ALU]
REG[Registres]
UC --> ALU
UC --> REG
end

subgraph "Bus système"
BUS_D["Bus de données<br/>(bidirectionnel)"]
BUS_A["Bus d'adresses<br/>(UC à mémoire)"]
BUS_C["Bus de contrôle<br/>(UC à périphériques)"]
end

subgraph "Mémoire et E/S"
RAM[RAM]
ENTREE[Entrée]
SORTIE[Sortie]
STOCK[Stockage]
end

REG <--> BUS_D
BUS_D <--> RAM
UC --> BUS_A
RAM --> BUS_A
UC --> BUS_C
BUS_C --> ENTREE
BUS_C --> SORTIE
BUS_C --> STOCK

style UC fill:#4a90d9,color:#fff
style ALU fill:#e67e22,color:#000
style REG fill:#9b59b6,color:#fff
style RAM fill:#2ecc71,color:#fff
style BUS_D fill:#e74c3c,color:#fff
style BUS_A fill:#e74c3c,color:#fff
style BUS_C fill:#e74c3c,color:#fff

```

Exercice 1 – Modèle de von Neumann (15 min)

Scénario enseignant : Le modèle de von Neumann est la base de l'architecture des ordinateurs modernes. Commencez par une analogie simple (recette de cuisine) puis faites remplir le tableau d'association.

Analogie : L'ordinateur est comme une cuisine :

- **ALU** = les ustensiles de cuisine (couteau, fouet, cuillère – les outils qui transforment les ingrédients)
- **UC** = le chef cuisinier qui lit la recette et dit quoi faire dans quel ordre
- **RAM** = le plan de travail où sont posés les ingrédients en cours d'utilisation
- **Disque dur** = le garde-manger/refrigérateur (stockage permanent)
- **Bus** = les mains et les déplacements du chef entre les postes de travail

– Question de base (6 min)

Nonc : Nommer les 5 unités du modèle de von Neumann et associer chaque composant à son rôle.

Composants à placer : ALU, Unité de contrôle, RAM, Disque dur, Clavier, Écran, Cache L1, Bus système.

Unité	Composant(s) associé(s)
Traitement	
Contrôle	
Mémoire	
Entrée	
Sortie	

Corrigé :

Unité	Composant(s)
Traitement	ALU
Contrôle	Unité de contrôle
Mémoire	RAM, Cache L1, Disque dur
Entrée	Clavier
Sortie	Écran

Le bus système relie toutes les unités entre elles : il transporte les adresses, les données et les signaux de contrôle.

... Question intermédiaire (5 min)

À noter : Compléter le schéma en remplaçant les 5 unités suivantes aux bons emplacements :

```

    à l'adresse 0x00000000, l'unité de contrôle est connectée à l'adresse 0x00000000.
    à l'adresse 0x00000001, l'unité de traitement est connectée à l'adresse 0x00000001.
    à l'adresse 0x00000002, l'unité de mémoire est connectée à l'adresse 0x00000002.
    à l'adresse 0x00000003, l'unité d'entrée est connectée à l'adresse 0x00000003.
    à l'adresse 0x00000004, l'unité de sortie est connectée à l'adresse 0x00000004.
    à l'adresse 0x00000005, l'unité de contrôle est connectée à l'adresse 0x00000005.
    à l'adresse 0x00000006, l'unité de traitement est connectée à l'adresse 0x00000006.
    à l'adresse 0x00000007, l'unité de mémoire est connectée à l'adresse 0x00000007.
    à l'adresse 0x00000008, l'unité d'entrée est connectée à l'adresse 0x00000008.
    à l'adresse 0x00000009, l'unité de sortie est connectée à l'adresse 0x00000009.
    à l'adresse 0x0000000A, l'unité de contrôle est connectée à l'adresse 0x0000000A.
    à l'adresse 0x0000000B, l'unité de traitement est connectée à l'adresse 0x0000000B.
    à l'adresse 0x0000000C, l'unité de mémoire est connectée à l'adresse 0x0000000C.
    à l'adresse 0x0000000D, l'unité d'entrée est connectée à l'adresse 0x0000000D.
    à l'adresse 0x0000000E, l'unité de sortie est connectée à l'adresse 0x0000000E.
    à l'adresse 0x0000000F, l'unité de contrôle est connectée à l'adresse 0x0000000F.
  
```

Corrigé : ALU (traitement), Unité de contrôle (contrôle), Clavier (entrée), RAM (mémoire), Écran (sortie).

... Question avancée (4 min)

À%noncÀ : Le modèle de von Neumann a un goulot d'étranglement célèbre : le **goulot de von Neumann** (von Neumann bottleneck). Expliquer de quoi il s'agit et proposer une solution architecturale pour l'atténuer.

CorrigÀ :

- **ProblÀme** : Le bus unique entre le CPU et la mémoire limite le débit (les instructions et les données empruntent le même chemin). Le CPU peut être plus rapide que le temps de transfert mémoire à l'attente des données.
- **Solutions** :
 - Architecture Harvard (bus séparés instructions/données)
 - Hiérarchie de caches (L1, L2, L3)
 - Mémoires plus rapides (DDR5, HBM)
 - Prédiction de branchement et exécution spéculative

Exercice 2 – Cycle d'exécution (15 min)

Scénario enseignant : Le cycle fetch-decode-execute-writeback est le battement de cœur du processeur. Faites-le mimer par les étudiants : un groupe mime le chargement, un autre le décodage, etc.

Question de base (5 min)

À%noncÀ : Remettre dans l'ordre les étapes du cycle d'exécution d'une instruction.

1. L'unité de contrôle décode l'instruction (opération + opérandes)
2. L'ALU exécute l'opération
3. Le résultat est écrit dans un registre ou en mémoire
4. L'instruction est chargée depuis la RAM vers le registre instruction

Ordre correct : 4 à 1 à 2 à 3 (Fetch à Decode à Execute à Write-back)

Application : Décrire ce qui se passe quand on tape A au clavier (depuis la pression de la touche jusqu'à l'affichage à l'écran).

Réponse : Clavier à interruption matérielle à UC à OS (lecture du scan code) à conversion en caractère à écriture en RAM à affichage à l'écran via le contrôleur graphique.

Question intermédiaire (5 min)

À%noncÀ : Trace d'exécution. On suppose l'instruction `ADD R1, R2, R3` (addition : $R1 \leftarrow R2 + R3$). Décrire chaque étape du cycle en détaillant les bus utilisés.

CorrigÀ :

Étape	Description
Fetch	L'UC envoie l'adresse de la prochaine instruction (PC) sur le bus d'adresses à la RAM renvoie l'instruction sur le bus de données
Decode	L'UC décode le code opération (ADD) et identifie les opérandes (R2, R3 comme sources, R1 comme destination)
Execute	L'ALU lit la valeur de R2 et R3 depuis les registres, effectue l'addition, stocke le résultat dans un registre temporaire

Write-back	Le résultat est écrit dans le registre R1. Le PC est incrémenté pour pointer vers l'instruction suivante
-------------------	--

Question avancée (5 min)

Nonc : Soit le programme machine suivant (langage d'assemblage simplifié) :

```
LOAD R1, #5 ; R1 ← 5 (chargement immédiat)
LOAD R2, #3 ; R2 ← 3
ADD R3, R1, R2 ; R3 ← R1 + R2
STORE R3, [100] ; Écrire R3 à l'adresse mémoire 100
HALT ; Arrêter
```

Exécuter le cycle complet pour **chaque instruction** et donner l'état final des registres (R1, R2, R3) et de la mémoire à l'adresse 100.

Corrigé :

Instruction	Fetch	Decode	Execute	Write-back	État PC
1: LOAD R1,#5	Charger instr. 1	LOAD immédiat	R1 ← 5	R1 = 5	PC = 2
2: LOAD R2,#3	Charger instr. 2	LOAD immédiat	R2 ← 3	R2 = 3	PC = 3
3: ADD R3,R1,R2	Charger instr. 3	ADD	R3 ← 5+3=8	R3 = 8	PC = 4
4: STORE R3,[100]	Charger instr. 4	STORE	Écrire R3 en mémoire	MEM[100] = 8	PC = 5
5: HALT	Charger instr. 5	HALT	Arrêter	-	PC = 5

État final : R1 = 5, R2 = 3, R3 = 8, MEM[100] = 8.

Exercice 3 – Hiérarchie mémoire (20 min)

Scénario enseignant : La hiérarchie mémoire est un compromis vitesse/coût. Utilisez l'image d'une pyramide : plus on monte, plus c'est rapide et cher, mais plus la capacité est faible.

Question de base (8 min)

Nonc : Classer les types de mémoire suivants du plus rapide au plus lent et du plus petit au plus grand : SSD 1 To, RAM 16 Go, Cache L1 64 Ko, Registres 512 o, HDD 2 To, Cache L2 512 Ko.

Corrigé :

Niveau	Type	Taille	Vitesse	Ordre de grandeur
1 (rapide)	Registres	512 o	< 1 ns	Comme une feuille de papier sur le bureau
2	Cache L1	64 Ko	~1 ns	Comme un tiroir de bureau
3	Cache L2	512 Ko	~4 ns	Comme une armoire dans la pièce
4	RAM	16 Go	~50 ns	Comme une bibliothèque dans le bâtiment
5	SSD	1 To	~100 µs	Comme un entrepôt en ville
6 (lent)	HDD	2 To	~10 ms	Comme un entrepôt dans une autre ville

Question : Pourquoi un ordinateur avec 8 Go de RAM peut-il être plus rapide qu'un autre avec 16 Go ?

Réponse : La vitesse ne dépend pas que de la RAM. Un CPU plus rapide (fréquence + cache), un SSD au lieu d'un HDD, et un OS optimisés peuvent compenser une RAM moindre.

Question intermédiaire (6 min)

Nonc : Principe de localité. On suppose qu'un programme accède 1000 fois consécutives à la même variable. Avec un cache L1 de 64 Ko et un temps d'accès de 1 ns (cache) vs 50 ns (RAM), calculer :

1. Le temps total sans cache (accès RAM à chaque fois).
2. Le temps total avec cache (1 accès RAM, 999 accès cache).
3. Le facteur d'accélération.

Corrigé :

1. **Sans cache** : $1000 \times 50 \text{ ns} = 50\,000 \text{ ns} = 50 \mu\text{s}$
2. **Avec cache** : $1 \times 50 \text{ ns}$ (premier accès, cache miss) + $999 \times 1 \text{ ns}$ (cache hit) = $50 + 999 = 1049 \text{ ns} \approx 1,05 \mu\text{s}$
3. **Facteur d'accélération** : $50\,000 / 1049 \approx 48$ — plus rapide !

Conclusion : Le cache exploite la localité temporelle (une variable accédée souvent reste dans le cache) et spatiale (les variables voisines sont chargées ensemble).

Question avancée (6 min)

Nonc : Une page web fait 4 Mo. Le cache L2 fait 512 Ko. Combien de « cache miss » (défauts de cache) y aura-t-il si la page est chargée intégralement via le cache ?

On suppose que le cache se remplit par blocs de 64 Ko (taille de ligne de cache).

Corrigé :

1. **Nombre total de blocs nécessaires** : $4 \text{ Mo} / 64 \text{ Ko} = 4096 \text{ Ko} / 64 \text{ Ko} = 64 \text{ blocs}$
2. **Capacité du cache L2** : $512 \text{ Ko} / 64 \text{ Ko} = 8 \text{ lignes de cache}$
3. **Cache miss** : le cache ne peut contenir que 8 blocs à la fois. Si la page est plus grande que le cache, chaque nouveau bloc provoque un miss. Avec un remplacement, il y aura au moins $64 - 8 = 56$ **cache misses** (dans le pire cas, 64 misses si les blocs ne se chevauchent pas).
4. **Ratio de hits** : $(64 - 56) / 64 = 12,5 \%$ de hits seulement dans ce cas d'où l'importance d'avoir un cache suffisamment grand pour les données fréquemment utilisées.

Exercice 4 – Ligne de commande (15 min)

Scénario enseignant : La ligne de commande est un outil puissant pour interagir avec l'OS. Cet exercice peut être fait sur machine si possible, ou à l'écrit. Corrigez en projetant les commandes.

Question de base (5 min)

Nonc : Écrire la commande Windows PowerShell pour :

1. Lister les fichiers du dossier courant
2. Créer un dossier `TD_Architecture`
3. Naviguer dans ce dossier
4. Créer un fichier vide `notes.txt`

5. Afficher l'adresse IP de la machine

Corrigé :

Action	Commande PowerShell
1. Lister	<code>Get-ChildItem</code> (ou <code>dir</code> , <code>ls</code>)
2. Créer dossier	<code>New-Item -ItemType Directory -Name "TD_Architecture"</code>
3. Naviguer	<code>Set-Location "TD_Architecture"</code> (ou <code>cd</code>)
4. Créer fichier	<code>New-Item -ItemType File -Name "notes.txt"</code>
5. Voir IP	<code>Get-NetIPAddress</code> (ou <code>ipconfig</code>)

Question intermédiaire (5 min)

Nonc : On veut créer l'arborescence suivante en ligne de commande :

```
C:\Exercices\
  "Cours\
    "architecture.txt
    "reseaux.txt
  "TP\
    "rendu.txt
```

Écrire la séquence de commandes PowerShell complète.

Corrigé :

```
New-Item -ItemType Directory -Name "Exercices"
Set-Location "Exercices"
New-Item -ItemType Directory -Name "Cours"
New-Item -ItemType Directory -Name "TP"
Set-Location "Cours"
New-Item -ItemType File -Name "architecture.txt"
New-Item -ItemType File -Name "reseaux.txt"
Set-Location "..\TP"
New-Item -ItemType File -Name "rendu.txt"
Set-Location "..\.."
```

Vérification : `Get-ChildItem -Recurse` pour afficher toute l'arborescence.

Question avancée (5 min)

Nonc : En une seule ligne PowerShell, trouver tous les fichiers `.txt` dans `C:\Users` et ses sous-dossiers, afficher leur nom, taille et date de modification, triés par taille décroissante.

Corrigé :

```
Get-ChildItem -Path "C:\Users" -Filter "*.txt" -Recurse |
Select-Object Name, Length, LastWriteTime |
Sort-Object -Property Length -Descending
```

Explication :

- `Get-ChildItem -Recurse` : parcourt récursivement
- `-Filter "*.txt"` : filtre les fichiers texte
- `Select-Object` : ne garde que les colonnes utiles
- `Sort-Object -Descending` : trie par taille décroissante

Variante plus concise : `gci -r "C:\Users" -fi ".txt" | sort len -desc | select name, len, last`

Exercice 5 – QCM Architecture (15 min)

Scénario enseignant : Ce QCM final permet de vérifier rapidement les acquis de la séance. Corrigez en grand groupe en demandant à chaque étudiant de justifier son choix.

Questions de base

À retenir : Choisir la bonne réponse.

1. Que signifie CPU ?

- a) Central Processing Unit
- b) Computer Personal Unit
- c) Central Program Utility

2. La mémoire RAM est :

- a) Non volatile
- b) Volatile
- c) Aussi rapide que les registres

3. Le modèle de von Neumann stocke :

- a) Uniquement les données
- b) Uniquement les programmes
- c) Les données ET les programmes

4. L'unité de contrôle fait partie :

- a) Du disque dur
- b) Du processeur
- c) De la RAM

5. La fréquence d'un CPU se mesure en :

- a) MHz ou GHz
- b) MB/s
- c) Watts

Questions intermédiaires

6. Quel est le rôle du bus d'adresses ?

- a) Transporter les données entre CPU et mémoire
- b) Transporter l'adresse de la donnée/instruction à lire
- c) Alimenter électriquement les composants

7. Le cache L1 est généralement :

- a) Plus grand que le cache L2

- b) Plus rapide que le cache L2
- c) Aussi lent que la RAM

8. La commande `Set-Location "Dossier"` sous PowerShell permet de :

- a) Créer un dossier
- b) Changer de dossier
- c) Supprimer un dossier

Question avancée

9. Si un CPU a une fréquence de 3 GHz, combien de cycles d'horloge s'écoulent en 1 nanoseconde ?

Rappel : $1 \text{ GHz} = 10^9 \text{ Hz} = 10^9 \text{ cycles par seconde}$

- a) 3 cycles
- b) 0,003 cycle
- c) 3×10^{-9} cycle

Corrigé : $3 \text{ GHz} = 3 \times 10^9 \text{ cycles/s}$ et $1 \text{ cycle} = 1/(3 \times 10^9) \text{ s} \approx 0,33 \text{ ns}$. Donc en 1 ns, on a 3 cycles. Réponse : a).

QCM d'auto-évaluation

1. Quelle unité du modèle de von Neumann effectue les calculs ?

- a) Unité de contrôle
- b) ALU
- c) Mémoire

2. Le rôle du cache est de :

- a) Stocker des fichiers
- b) Accélérer l'accès aux données fréquentes
- c) Refroidir le CPU

3. Le cycle d'exécution commence par l'étape :

- a) Execute
- b) Decode
- c) Fetch

4. Parmi ces mémoires, laquelle est la plus rapide ?

- a) RAM
- b) Registres
- c) SSD

5. La commande `Get-ChildItem` sous PowerShell permet de :

- a) Lister les fichiers
- b) Créer un dossier
- c) Effacer un fichier

6. L'architecture de von Neumann est utilisée par :

- a) Tous les ordinateurs modernes
- b) Uniquement les PC
- c) Aucun ordinateur

7. Le principe de localité spatiale signifie que :

- a) Les instructions voisines seront probablement exécutées ensemble
- b) Une variable sera réutilisée
- c) Le code est rangé par taille

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Schéma von Neumann	20 %	Schéma inexistant ou faux	Schéma partiel	Schéma complet et correct
Cycle Fetch-Decode-Execute	20 %	Étapes dans le désordre	3 étapes sur 4	4 étapes correctes dans l'ordre

Hiérarchie mémoire	20 %	Ordre incorrect	3 niveaux sur 5	5 niveaux corrects
Registres CPU	20 %	Aucun registre identifié	2-3 registres	4+ registres corrects
Analogies pédagogiques	20 %	Aucune analogie proposée	Analogie partielle	Analogie pertinente et justifiée

Note pédagogique ENS

Transposition didactique

Le modèle de von Neumann peut être introduit en collège/lycée avec une analogie simple : le cerveau (CPU) qui lit une recette de cuisine (programme en mémoire), exécute les instructions (ALU), et utilise le plan de travail (RAM) pour stocker les ingrédients temporaires. Chaque élève peut mimer le rôle d'un composant (mémoire, ALU, contrôleur) dans une activité théâtrale.

Conseil pour l'enseignant formateur

L'exercice 2 (cycle d'exécution) est fondamental : s'assurer que chaque étudiant sait réciter les 4 étapes dans l'ordre. Utiliser un jeu de cartes ou de flashcards pour mémoriser le cycle. Pour la hiérarchie mémoire, insister sur les ordres de grandeur (ns vs μ s vs ms) qui sont abstraits pour les débutants. L'exercice de trace d'exécution (à...à...) est très formateur mais peut être réservé aux étudiants les plus avancés.

Prolongement

Proposer aux étudiants de démonter (virtuellement via des schémas ou réellement avec un vieux ordinateur) les composants d'un PC et de les identifier. Réaliser une affiche de classe représentant le modèle de von Neumann avec les composants réels d'un ordinateur du laboratoire.

Pour aller plus loin

- **Cycle d'exécution** : Simuler le cycle avec un jeu de cartes où chaque carte représente une instruction machine.
- **Hiérarchie mémoire** : Demander aux étudiants de chronométrer différents accès (lecture d'un fichier sur HDD vs SSD vs RAM disk).
- **Ligne de commande** : Organiser un défi « course aux commandes » : l'enseignant donne une tâche, le premier étudiant qui écrit la commande correcte gagne un point.
- **QCM** : Les étudiants créent eux-mêmes 3 questions d'architecture pour leurs camarades (format pair-impair).

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_1ère_année_FR_04.pdf

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 04 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- Expliquer le rÃ´le du systÃ©me d'exploitation et de ses composants (noyau, pilotes)
- DÃ©crire les Ã©tats d'un processus et leur cycle de vie
- Naviguer dans une arborescence de fichiers et construire des chemins absolus
- ExÃ©cuter des commandes PowerShell pour manipuler fichiers et dossiers
- Distinguer les privilÃ©ges noyau/mode utilisateur via les appels systÃ©me

Contexte pÃ©dagogique â€” ScÃ©nario enseignant

Vous Ãªtes enseignant et devez initier vos Ã©tudiants au **rÃ´le central du systÃ©me d'exploitation**. Ce TD explore les couches internes d'un OS (noyau, pilotes, syscalls), le cycle de vie des processus, l'arborescence des fichiers et la ligne de commande PowerShell. Utilisez l'**analogie du chef d'orchestre** (OS) qui dirige les musiciens (processus) et distribue les partitions (ressources).

Symbole	Niveau	Attendu
â€œ	DÃ©butant	Questions de base â€” tous les Ã©tudiants doivent rÃ©pondre
âˆ™...	IntermÃ©diaire	Approfondissement â€” Ã traiter si le temps le permet
âˆ™...âˆ™...	AvancÃ©	DÃ©fi â€” pour les plus rapides, peut faire l'objet d'un bonus

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” RÃ´le de l'OS	15 min	20 min
Exercice 2 â€” Processus	15 min	35 min
Exercice 3 â€” Arborescence	20 min	55 min
Exercice 4 â€” PowerShell	15 min	70 min
Exercice 5 â€” Syscalls	15 min	85 min
Correction	5 min	90 min
Total	90 min	

Architecture d'un syst me d'exploitation

```

graph TB
    subgraph "Utilisateur"
        APP1[Application 1]
        APP2[Application 2]
        APP3[Application 3]
    end

    subgraph "Mode Utilisateur"
        APP1 --> API[API / Appels syst me]
        APP2 --> API
        APP3 --> API
    end

    subgraph "Mode Noyau"
        API --> Noyau[Noyau<br/>Kernel]
        Noyau --> GProc[Gestion des processus]
        Noyau --> GMem[Gestion m moire]
        Noyau --> GFich[Gestion fichiers]
        Noyau --> GPilotes[Pilotes / Drivers]
    end

    subgraph "Mat riel"
        GPilotes --> CPU
        GPilotes --> RAM
        GPilotes --> DD[Disque dur]
        GPilotes --> Res[R seau]
    end

    style APP1 fill:#4a90d9,color:#fff
    style APP2 fill:#4a90d9,color:#fff
    style APP3 fill:#4a90d9,color:#fff
    style Noyau fill:#e74c3c,color:#fff
    style GProc fill:#2ecc71,color:#fff
    style GMem fill:#2ecc71,color:#fff
    style GFich fill:#2ecc71,color:#fff
    style GPilotes fill:#2ecc71,color:#fff
    style CPU fill:#e67e22,color:#000
    style RAM fill:#e67e22,color:#000
    style DD fill:#e67e22,color:#000
    style Res fill:#e67e22,color:#000

```

Cycle de vie d'un processus

```

stateDiagram-v2
    [*] --> Pr t : Cr ation / Chargement
    Pr t -->  lu : Ordonnancement (scheduler)
     lu --> Pr t : Interruption / Fin du quantum
     lu --> Bloqu  : Appel syst me bloquant (E/S)
    Bloqu  --> Pr t : Fin de l'attente (E/S termin e)
     lu --> [*] : Terminaison

    note right of Pr t
        File d'attente des
        processus pr ts
    end note

    note right of  lu
        Le CPU ex cute
        les instructions
    end note

```

```

note right of Bloqu  
Attente d'une ressource
(disque, r  seau, clavier...)
end note

```

Tableau des permissions Unix (r  f  rence)

Symbole	Permission	Valeur	Signification
---	Aucune	0	Pas de droit
--x	Ex��cution	1	Droit d'ex��cuter (fichier) ou traverser (dossier)
-w	��criture	2	Droit de modifier
-wx	��criture + Ex��cution	3	Modifier + ex��cuter
r--	Lecture	4	Droit de lire
r-x	Lecture + Ex��cution	5	Lire + ex��cuter
rw	Lecture + ��criture	6	Lire + modifier
rwx	Tous les droits	7	Lecture, ��criture, ex��cution

Structure : `-rwxr-xr--` se d  compose en :

- `-` : type (fichier normal, `d` = dossier)
- `rwx` : droits du propri  taire (user)
- `r-x` : droits du groupe (group)
- `r--` : droits des autres (others)

Exercice 1   ” R  le de l'OS (15 min)

Sc  nario enseignant : L'objectif est de faire comprendre que l'OS est un **interm  diaire obligatoire** entre les applications et le mat  riel. Sans OS, chaque application devrait g  rer elle-m  me les pilotes, l'ordonnancement, etc.

  ” Question de base (6 min)

  nonc   : Associer chaque fonction    sa description.

Fonction	Description
Gestion des processus	A. Organiser les fichiers en arborescence
Gestion de la m��moire	B. Traduire les appels syst��me vers le mat��riel
Gestion des fichiers	C. Allouer la RAM aux applications
Noyau	D. Ordonnancer l'ex��cution des programmes
Pilote (driver)	E. Interface entre l'OS et un p��riph��rique

Corrig   :

Fonction	Description
----------	-------------

Gestion des processus	D " Ordonnancer l'exécution des programmes
Gestion de la mémoire	C " Allouer la RAM aux applications
Gestion des fichiers	A " Organiser les fichiers en arborescence
Noyau	B (partiellement) " C "ur du syst"me, g"re tout
Pilote (driver)	E " Interface entre l'OS et un p"riph"rique

â... Question interm"diaire (5 min)

Â%nonc" : Classer les actions suivantes selon qu'elles rel"vent du **noyau** (mode noyau) ou d'une **application** (mode utilisateur) :

1. Un calcul dans un tableur Excel
2. L"criture d'un fichier sur le disque dur
3. L'affichage d'une image " l"cran (via le pilote graphique)
4. La lecture d'une touche du clavier
5. L'ordonnancement du prochain processus " ex"cuter
6. La navigation dans un navigateur web

Corrig" :

Action	Mode
Calcul dans Excel	Utilisateur (purement applicatif)
Â"criture sur disque	Noyau (via syscall <code>write</code>)
Affichage via pilote graphique	Noyau (le pilote est dans l'OS)
Lecture clavier	Noyau (interruption mat"rielle g"r"e par l'OS)
Ordonnancement	Noyau (c'est le scheduler)
Navigation web	Utilisateur (le navigateur fait des syscalls mais reste en mode utilisateur)

â...â... Question avanc"e (4 min)

Â%nonc" : Lister 3 diff"rences entre un noyau **monolithique** (Linux) et un noyau **hybride** (Windows). Donner un avantage et un inconv"nient pour chaque approche.

Corrig" :

Caract"ristique	Monolithique (Linux)	Hybride (Windows)
Architecture	Tout le noyau dans un seul espace d'adressage	Noyau minimal + services en mode utilisateur
Performance	Meilleure (moins de changements de contexte)	L"g"rement moins bonne (IPC entre mod)
Stabilit" / S"curit"	Moins stable (un pilote crash tout le noyau)	Plus stable (un pilote crash peut "tre isol"
Modularit"	Chargeable via modules (<code>.ko</code>)	Services s"par"s

Avantage monolithique : performances "lev"es. **Inconv"nient** : risque de planter tout le syst"me.

Avantage hybride : meilleure isolation (un pilote qui plante ne bloque pas tout). **Inconvénient** : surcoût de communication entre les couches.

Exercice 2 – Processus et États (15 min)

Scénario enseignant : Le diagramme d'états d'un processus est central en OS. Faites-le reproduire de mémoire par les étudiants (d'abord avec les 3 États, puis les transitions). Utilisez un jeu de rôle : 3 chaises = Prêt, Exécuté, Bloqué ; des étudiants mimant les transitions.

Question de base (5 min)

Question : Un processus passe par 3 États : Prêt, Exécuté, Bloqué. Associer chaque situation à l'État correspondant.

1. Un processus attend que l'utilisateur clique sur un bouton.
2. Le CPU exécute les instructions du processus.
3. Le processus est dans la file d'attente, prêt à être exécuté.
4. Un processus lit un fichier sur le disque dur.
5. Le système d'exploitation vient de charger le processus en mémoire.

Corrigé :

1. **Bloqué** : attente d'une interaction utilisateur (E/S)
2. **Exécuté** : le CPU exécute activement le processus
3. **Prêt** : en attente dans la file d'ordonnancement
4. **Bloqué** : attente de la fin de l'opération disque (E/S bloquante)
5. **Prêt** : le processus est chargé mais pas encore exécuté

Question intermédiaire (5 min)

Question : On observe un système avec 3 processus (P1, P2, P3). L'ordonnanceur utilise le **Round Robin** avec un quantum de 2 unités de temps. Compléter le diagramme de Gantt sachant que les processus arrivent tous à $t = 0$ et qu'aucun ne fait d'E/S :

Temps	0-2	2-4	4-6	6-8	8-10	10-12	12-14	14-16
CPU	?	?	?	?	?	?	?	?

Corrigé :

Round Robin : P1 → P2 → P3 → P1 → P2 → P3 → P1 → ...

Temps	0-2	2-4	4-6	6-8	8-10	10-12	12-14	14-16
CPU	P1	P2	P3	P1	P2	P3	P1	P2

Explication : Chaque processus reçoit 2 unités de temps. Le cycle P1 → P2 → P3 se répète. L'ordre d'arrivée détermine l'ordre initial.

Question avancée (5 min)

À%noncÀ : On reprend le mÃame systÃme mais P2 fait une E/S bloquante au temps t=3 (durÃe : 3 unitÃs). DÃcrire l'Ãtat de chaque processus aux instants t=0, t=2, t=4, t=6, t=8.

CorrigÀ :

Temps	P1	P2	P3	ExÀ
t=0	PrÃt	PrÃt	PrÃt	â€
t=2	Ã%lu (0-2)	PrÃt	PrÃt	P1
t=3	PrÃt	Ã%lu (2-3) puis BloquÃ	PrÃt	P2 (pu
t=4	PrÃt	BloquÃ	Ã%lu (3-4, fin de quantum)	P3
t=6	Ã%lu (4-6)	BloquÃ	PrÃt	P1
t=8	PrÃt	PrÃt (E/S terminÃe Ã t=6, repasse en file)	Ã%lu (6-8)	P3

Observation : Quand P2 est bloquÃ, l'ordre Round Robin continue entre P1 et P3 jusqu'au retour de P2.

Exercice 3 â€” Arborescence des fichiers (20 min)

ScÃnario enseignant : L'arborescence est le modÃle standard d'organisation des fichiers. Les Ãtudiants doivent maÃtriser les chemins absolus et relatifs, la notion de dossier parent/enfant, et savoir construire une arborescence Ã partir d'une description textuelle.

â€ Question de base (8 min)

À%noncÀ : Dessiner l'arborescence correspondant au chemin suivant :

```
C:\
  âœ€â€ Utilisateurs\
    â", â""â€ Madani\
      â", âœ€â€ Documents\
        â", â", âœ€â€ Cours_ENS1\
          â", â", â""â€ TD_Info.txt
        â", âœ€â€ Images\
          â", â""â€ Bureau\
            â", â""â€ Projet_Python\
              âœ€â€ Program Files\
                â", â""â€ Python3\
                  â""â€ Windows\
                    â""â€ System32\
```

Questions :

1. Quel est le chemin absolu vers TD_Info.txt ?
2. Quel est le dossier parent de Projet_Python ?
3. Combien de dossiers contient directement C : \ (dossiers de premier niveau) ?
4. Quel est le chemin relatif de TD_Info.txt vers Projet_Python ?

CorrigÀ :

1. C:\Utilisateurs\Madani\Documents\TD_Info.txt
2. Bureau (car Bureau\Projet_Python)
3. 3 dossiers : Utilisateurs, Program Files, Windows

4. `..\..\Bureau\Projet_Python` (remonter de Documents vers Madani, puis vers Utilisateurs, puis descendre dans Bureau\Projet_Python... en fait : `../../Bureau/Projet_Python`)

~... Question intermédiaire (6 min)

Nonc : Construire l'arborescence qui correspond aux spécifications suivantes :

- Racine : `D:\Scolarite`
- Sous `Scolarite` : 2 dossiers `Cours` et `Examens`
- `Cours` contient : `Informatique` (sous-dossier) + `Mathematiques` (sous-dossier) + `EmploiDuTemps.txt`
- `Informatique` contient : `TDs` (sous-dossier) + `Projets` (sous-dossier)
- `TDs` contient : `TD01.pdf` et `TD02.pdf`
- `Projets` contient : `Projet_Python.zip`
- `Examens` contient : `Partiel_Info.pdf`

Dessiner l'arborescence et donner le chemin absolu de `Projet_Python.zip`.

Corrigé :

```
D:\Scolarite\
  "Cours\
    "Informatique\
      "TDs\
        "TD01.pdf
        "TD02.pdf
      "Projets\
        "Projet_Python.zip
    "Mathematiques\
      "EmploiDuTemps.txt
  "Examens\
    "Partiel_Info.pdf
```

Chemin absolu de `Projet_Python.zip` :
`D:\Scolarite\Cours\Informatique\Projets\Projet_Python.zip`

~... Question avancée (6 min)

Nonc : On se place dans le dossier `Cours`. Pour chaque chemin relatif, donner le chemin absolu correspondant :

1. `..\Examens\Partiel_Info.pdf`
2. `Informatique\..\Mathematiques`
3. `Informatique\TDs\..\..\Examens\Partiel_Info.pdf`
4. . (point seul)

Corrigé :

1. `D:\Scolarite\Examens\Partiel_Info.pdf` (remonter de `Cours` à `Scolarite` puis descendre dans `Examens`)
2. `D:\Scolarite\Cours\Mathematiques` (`Informatique` puis `..` remonte dans `Cours`, puis `Mathematiques`)

3. D:\Scolarite\Examens\Partiel_Info.pdf (Informatique â†’ TDs â†’ .. = Informatique â†’ .. = Cours â†’ .. = Scolarite â†’ Examens)

4. D:\Scolarite\Cours (le point représente le dossier courant)

Tableau d'entrÃ©nement â€” Chemins absolus et relatifs

Position actuelle	Cible	Chemin
C:\Users\Madani\Docs	C:\Users\Madani\Images\photo.jpg	<font fac
C:\Windows\System32	C:\Windows\notepad.exe	
D:\Projets\Python	D:\Data\cours.pdf	
C:\	C:\Users\Public\Desktop\readme.txt	

CorrigÃ© :

- ..\Images\photo.jpg (remonter dans Madani, descendre dans Images)
- ..\notepad.exe (remonter dans Windows)
- ...Data\cours.pdf (remonter dans Projets, puis D:\, descendre dans Data)
- Users\Public\Desktop\readme.txt

Exercice 4 â€” Commandes PowerShell (15 min)

ScÃ©nario enseignant : Les commandes PowerShell sont essentielles pour la maintenance systÃ©me. Si possible, ouvrez PowerShell en direct et faites exÃ©cuter les commandes par les Ã©tudiants. Sinon, faites-les Ã© exÃ©cuter mentalement Ã© en dÃ©crivant le rÃ©sultat.

â€œ Question de base (5 min)

Ã©noncÃ© : ExÃ©cuter mentalement la sÃ©quence de commandes suivante et dÃ©crire le rÃ©sultat.

```
New-Item -ItemType Directory -Name "Stage"
Set-Location "Stage"
New-Item -ItemType File -Name "Rapport.txt"
Set-Content -Path "Rapport.txt" -Value "Mon rapport de stage"
Get-ChildItem
Remove-Item -Path "Rapport.txt"
Get-Location
```

CorrigÃ© :

1. CrÃ©e le dossier Stage
2. Se place dans Stage
3. CrÃ©e le fichier Rapport.txt (vide)
4. Ã©crit Ã© Mon rapport de stage Ã© dans le fichier (remplace le contenu)
5. Liste le contenu du dossier (affiche Rapport.txt)
6. Supprime le fichier Rapport.txt
7. Affiche le chemin courant (... \Stage)

RÃ©sultat final : Le dossier Stage existe mais est vide. Le terminal est positionnÃ© dans Stage.

â€¦ Question intermédiaire (5 min)

Â€noncÂ€ : Â€crire une sÂ€quence PowerShell qui rÂ€alise les opÂ€rations suivantes dans l'ordre :

1. Depuis C:\, crÂ€er la structure C:\ENS\TD\OS
2. CrÂ€er un fichier `reponses.txt` dans OS
3. Ajouter (sans effacer) la ligne Â€« Exercice 1 terminÂ€ Â€» dans `reponses.txt`
4. Afficher le contenu du fichier
5. Copier `reponses.txt` vers C:\ENS\reponses_backup.txt

CorrigÂ€ :

```
New-Item -ItemType Directory -Path "C:\ENS\TD\OS" -Force # -Force crÂ€e toute la chaÂ€ne
Set-Content -Path "C:\ENS\TD\OS\reponses.txt" -Value "Exercice 1 terminÂ€"
Get-Content -Path "C:\ENS\TD\OS\reponses.txt"
Copy-Item -Path "C:\ENS\TD\OS\reponses.txt" -Destination "C:\ENS\reponses_backup.txt"
```

Astuce : `Add-Content` permet d'ajouter (append) au lieu d'Â€craser.

Exemple : `Add-Content -Path "fichier.txt" -Value "Nouvelle ligne"`

â€¦ Question avancÂ€e (5 min)

Â€noncÂ€ : Â€crire un script PowerShell qui parcourt rÂ€cursivement un dossier donnÂ€ en argument et affiche pour chaque fichier :

- Son nom
- Sa taille en Ko
- Sa date de derniÂ€re modification
- L'extension

Trier les rÂ€sultats par taille dÂ€croissante et n'afficher que les 10 plus gros fichiers.

CorrigÂ€ :

```
param($Dossier = ".")
Get-ChildItem -Path $Dossier -Recurse -File |
Select-Object Name, @{N="TailleKo";E={[math]::Round($_.Length/1KB, 2)}},
LastWriteTime, Extension |
Sort-Object -Property TailleKo -Descending |
Select-Object -First 10
```

Explication :

- `param($Dossier = ".")` : paramÂ€tre avec valeur par dÂ€faut (dossier courant)
- `-Recurse -File` : parcourt rÂ€cursivement, ne garde que les fichiers
- `Select-Object` avec `@{...}` : crÂ€e une propriÂ€tÂ€ calculÂ€e `TailleKo`
- `[math]::Round(...)` : arrondit Â 2 dÂ€cimales
- `Sort-Object -Descending` : tri dÂ€croissant
- `-First 10` : top 10

4. **Valeur octale : 754** ($rwx=7$, $r-x=5$, $r--=4$).

Question avancée (5 min)

Scénario pratique. Un étudiant A crée un script `evaluation.py` dans son dossier personnel avec les permissions `-rw-----`. Un étudiant B veut l'exécuter.

1. Pourquoi B ne peut-il pas exécuter le script ?
2. Que doit faire A pour que B puisse l'exécuter (sans pouvoir le modifier) ?
3. Et pour que tout le monde puisse le lire ET l'exécuter ?
4. Quelle commande PowerShell (sous Windows) permet de gérer les permissions ? Quelle commande Linux fait la même chose ?

Corrigé :

1. **Permissions** : `-rw-----` = 600. Le propriétaire (A) a lecture+écriture. Personne d'autre n'a aucun droit. B ne peut ni lire, ni exécuter.
2. **Donner l'exécution au groupe** : `chmod 710 evaluation.py` (ou `chmod g+rx evaluation.py`). `â†' -rwx--x---` (propriétaire : tout, groupe : exécution uniquement). B peut exécuter mais pas lire le code source.
3. **Lecture + exécution pour tous** : `chmod 755 evaluation.py â†' -rwxr-xr-x`. Tout le monde peut lire et exécuter. La valeur 755 est le standard pour les scripts publics.
4. **Windows (PowerShell)** : `Get-Acl / Set-Acl` pour lire/modifier les permissions (ACL).

Linux : `chmod`, `chown`, `chgrp` pour gérer les permissions.

Synthèse visuelle de "Appel système (syscall)"

```
sequenceDiagram
    participant App as Application (mode utilisateur)
    participant OS as Noyau (mode noyau)
    participant HW as Matériel

    App->>App: Calcul interne (pas de syscall)
    App->>OS: syscall : open("fichier.txt")
    Note over App,OS: Changement de contexte utilisateur â†' noyau
    OS->>HW: Lecture secteur disque
    HW-->>OS: Données brutes
    OS-->>App: Descripteur de fichier (fd)
    Note over App,OS: Retour en mode utilisateur
    App->>App: Traitement des données
```

QCM d'auto-évaluation

1. **Le noyau d'un système d'exploitation est :**

a) Un logiciel applicatif b) Le cœur du système qui gère les ressources c) Un pilote de périphérique

2. **Un processus l'état « Bloqué » attend :**

a) Que le CPU soit libre b) Une ressource (E/S, clavier...) c) La fin du quantum de temps

3. **Dans un chemin `C:\Users\Nom\Doc\fichier.txt`, le dossier parent de `Doc` est :**

a) `C:\` b) `Nom` c) `Users`

4. Un appel système (syscall) permet de :

- a) Redémarrer l'ordinateur b) Passer du mode utilisateur au mode noyau c) Installer un logiciel

5. La commande PowerShell `Remove-Item` permet de :

- a) Créer un fichier b) Supprimer un fichier c) Déplacer un fichier

6. Les permissions `-rwxr-----` correspondent à la valeur octale :

- a) 740 b) 750 c) 700

7. Dans l'ordonnancement Round Robin, le quantum de temps est :

- a) Le temps total d'exécution b) La durée maximale allouée à chaque processus avant bascule c) La durée d'une E/S

8. Un chemin commençant par `..\..\Dossier\` signifie :

- a) Remonter de 2 niveaux puis descendre dans Dossier b) Descendre de 2 niveaux c) Aller à la racine

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Rôle du noyau	20 %	Ne connaît pas le rôle	Rôle partiellement compris	Rôle bien compris
État d'un processus	20 %	État dans le désordre	2-3 États corrects	5 États corrects
Ordonnancement Round Robin	20 %	Ne comprend pas le principe	Principe partiellement compris	Principe bien compris
Arborescence fichiers	20 %	Structure incorrecte	Structure partiellement correcte	Structure correcte
Ligne de commande	20 %	Aucune commande correcte	2-3 commandes correctes	5+ commandes correctes

Note pédagogique ENS**Transposition didactique**

La notion de système d'exploitation peut être introduite en SNT Seconde avec l'analogie du chef d'orchestre (OS) qui dirige les musiciens (applications) et distribue les partitions (ressources). L'arborescence des fichiers est un concept très concret : les étudiants peuvent explorer l'explorateur Windows pour retrouver les dossiers vus en TD. Pour les commandes PowerShell, organiser un défi « course aux commandes » où chaque groupe doit accomplir une série de tâches le plus rapidement possible.

Conseil pour l'enseignant formateur

L'exercice 3 (arborescence) est un bon support pour vérifier que les étudiants maîtrisent la différence entre chemin absolu et relatif. Proposer des exercices supplémentaires où les étudiants doivent écrire le chemin relatif entre deux points de l'arborescence. Pour les États du processus, le diagramme en machine à États est à faire reproduire par les étudiants de mémoire. L'ordonnancement Round Robin (à...à...) peut faire l'objet d'une activité physique où les étudiants se passent un ballon (le CPU) à tour de rôle.

Prolongement

Demander aux étudiants d'installer une machine virtuelle Linux (Ubuntu) sous VirtualBox et de reproduire les mêmes commandes en terminal bash : `ls`, `mkdir`, `cd`, `touch`, `cat`, `rm`, `pwd`. Comparer

les syntaxes PowerShell/Linux et discuter des différences philosophiques entre les deux OS.

Pour aller plus loin

- **Étatats de processus** : Faire un jeu de rôle avec 3 chaises (Prat, Alu, Bloqué) et des étudiants qui jouent les processus.
- **Permissions** : Sur une machine Linux, faire `ls -l` dans `/usr/bin` et interpréter les permissions de 3 fichiers.
- **Arborescence** : Dessiner l'arborescence de son propre `C:\Users` (sans entrer dans les sous-dossiers système).
- **PowerShell** : Demander aux étudiants d'écrire un script qui crée automatiquement leur arborescence de fichiers personnelle.
- **Syscalls** : Sur Linux, utiliser `strace ls` pour voir les appels système émis par une commande simple.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_FR_05.pdf`

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 05 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- Distinguer les types de donnÃ©es (entier, rÃ©el, boolÃ©en, chaÃªne) et les utiliser correctement
- Ã‰crire des algorithmes utilisant des structures conditionnelles (si/sinon) et des boucles (pour, tant que)
- Concevoir un algorithme complet de calcul de moyenne avec recherche du min/max
- Estimer et comparer la complexitÃ© algorithmique de diffÃ©rents algorithmes ($O(1)$, $O(n)$, $O(n^2)$)
- Analyser un problÃ©me et le dÃ©composer en Ã©tapes algorithmiques

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” Variables et types	15 min	20 min
Exercice 2 â€” Conditions	20 min	40 min
Exercice 3 â€” Boucles	20 min	60 min
Exercice 4 â€” Algorithme complet	20 min	80 min
Exercice 5 â€” ComplexitÃ©	10 min	90 min
Total	>90 min	

Structures algorithmiques fondamentales

```

flowchart TD
subgraph "Types de donnÃ©es"
T1["int : 42, -5"]
T2["float : 3.14, 0.001"]
T3["bool : vrai, faux"]
T4["string : 'Bonjour'"]
end

subgraph "Structures de contrÃ´le"
C1["Condition<br/>SI ... ALORS ... SINON"]
C2["Boucle POUR<br/>Nombre fixe d'itÃ©rations"]
C3["Boucle TANT QUE<br/>Condition d'arrÃªt"]
end

subgraph "Algorithmes"
```

```

A1[Calcul somme/moyenne]
A2[Recherche min/max]
A3[Table de multiplication]
end

T1 --> C1
T2 --> C1
T3 --> C1
T4 --> C1
C1 --> A1
C1 --> A2
C2 --> A1
C2 --> A2
C2 --> A3
C3 --> A1

style T1 fill:#4a90d9,color:#fff
style T2 fill:#4a90d9,color:#fff
style T3 fill:#4a90d9,color:#fff
style T4 fill:#4a90d9,color:#fff
style C1 fill:#e67e22,color:#000
style C2 fill:#e67e22,color:#000
style C3 fill:#e67e22,color:#000
style A1 fill:#2ecc71,color:#fff
style A2 fill:#2ecc71,color:#fff
style A3 fill:#2ecc71,color:#fff

```

Comparaison des complexités

```

graph LR
    subgraph "Complexité"
        O1["O(1) â€” Constant<br/>Accès direct par indice"]
        On["O(n) â€” Linéaire<br/>Parcours d'un tableau"]
        On2["O(n²) â€” Quadratique<br/>Double boucle imbriquée"]
    end

    O1 --> Ex1["Ex: Lire tab[3]"]
    On --> Ex2["Ex: Rechercher un nom"]
    On2 --> Ex3["Ex: Comparer chaque élément<br/>avec tous les autres"]

    style O1 fill:#2ecc71,color:#fff
    style On fill:#e67e22,color:#000
    style On2 fill:#e74c3c,color:#fff
    style Ex1 fill:#4a90d9,color:#fff
    style Ex2 fill:#4a90d9,color:#fff
    style Ex3 fill:#4a90d9,color:#fff

```

Exercice 1 â€” Variables et types (15 min)

Nonc : Identifier le type (entier, réel, booléen, chaîne) de chaque valeur :

- 42
- "Bonjour"
- 3.14
- vrai
- 'A'
- 5

7. 0.001

8. "123"

Corrigé :

1. Entier (int)
2. Chaîne (string)
3. Réel (float)
4. Booléen (bool)
5. Chaîne (string) « un caractère seul reste une chaîne
6. Entier (int)
7. Réel (float)
8. Chaîne (string) « les guillemets indiquent une chaîne même si le contenu est numérique

Q : Quel est le résultat de `int(3.14)` ? De `float("42")` ?

• **R :** `int(3.14) = 3` (troncature). `float("42") = 42.0`

Exercice 2 « Conditions (20 min)

Nononcé : Écrire en pseudo-code un algorithme qui :

1. Demande l'âge de l'utilisateur
2. Affiche « Majeur » si l'âge ≥ 18 , « Mineur » sinon
3. Si l'âge ≥ 65 , afficher « Senior » en plus

Corrigé :

```

AFFICHER "Quel est votre âge ?"
LIRE age
SI age >= 18 ALORS
  AFFICHER "Majeur"
SI age >= 65 ALORS
  AFFICHER "Senior"
FIN SI
SINON
  AFFICHER "Mineur"
FIN SI

```

Code Python Équivalent

```

age = int(input("Quel est votre âge ? "))
if age >= 18:
    print("Majeur")
if age >= 65:
    print("Senior")
else:
    print("Mineur")

```

Variante « Classification complète

```

SI age < 13 ALORS
  AFFICHER "Enfant"
SINON SI age < 18 ALORS
  AFFICHER "Adolescent"

```

```

SINON SI age < 65 ALORS
AFFICHER "Adulte"
SINON
AFFICHER "Senior"
FIN SI

```

Table de vérité :

Age	Affichage
10	Mineur
18	Majeur
65	Majeur, Senior

Exercice 3 – Boucles (20 min)

Nonc : écrire en pseudo-code :

1. Afficher les nombres pairs de 2 à 20 (inclus)
2. Calculer la somme des nombres de 1 à n (n saisi par l'utilisateur)
3. Afficher la table de multiplication d'un nombre saisi (de 1 à 10)

Corrigé :

```

// 1. Nombres pairs
POUR i DE 2 À 20 PAS 2 FAIRE
AFFICHER i
FIN POUR

// 2. Somme 1 à n
AFFICHER "Donner n"
LIRE n
somme ← 0
POUR i DE 1 À n FAIRE
somme ← somme + i
FIN POUR
AFFICHER "Somme : ", somme

// 3. Table de multiplication
AFFICHER "Donner un nombre"
LIRE nb
POUR i DE 1 À 10 FAIRE
AFFICHER nb, " × ", i, " = ", nb * i
FIN POUR

```

Schéma de l'algorithme (somme 1 à n)

```

flowchart TD
    A[Début] --> B[Afficher 'Donner n']
    B --> C[Lire n]
    C --> D[somme ← 0]
    D --> E[i ← 1]
    E --> F{i ≤ n ?}
    F -->|Oui| G[somme ← somme + i]
    G --> H[i ← i + 1]
    H --> F
    F -->|Non| I[Afficher somme]
    I --> J[Fin]

```

```

style A fill:#4a90d9,color:#fff
style J fill:#4a90d9,color:#fff
style D fill:#e67e22,color:#000
style G fill:#e67e22,color:#000
style F fill:#e74c3c,color:#fff
style I fill:#2ecc71,color:#fff

```

Table de traçage (n = 5)

i	somme	i % 5 ?
1	1	Vrai
2	3	Vrai
3	6	Vrai
4	10	Vrai
5	15	Vrai
6	â€œ	Faux â†’ fin

Résultat : somme = 15

Code Python Équivalent

```

# 1. Nombres pairs de 2 à 20
for i in range(2, 21, 2):
    print(i)

# 2. Somme 1 à n
n = int(input("Donner n : "))
somme = 0
for i in range(1, n + 1):
    somme += i
print("Somme :", somme)

# 3. Table de multiplication
nb = int(input("Donner un nombre : "))
for i in range(1, 11):
    print(nb, "×", i, "=", nb * i)

```

Variante TANT QUE : remplacer POUR i DE 1 À n par i ← 1 / TANT QUE i ≤ n / i ← i + 1.

Exercice 4 « Algorithme complet (20 min)

À noter : Écrire un algorithme qui :

1. Demande 5 notes à l'utilisateur
2. Calcule la moyenne
3. Affiche « Admis » si moyenne ≥ 10, « Ajourné » sinon
4. Affiche la note la plus haute et la plus basse

Corrigé :

```

max ← 0
min ← 20
somme ← 0

```

```

POUR i DE 1 À 5 FAIRE
  AFFICHER "Note ", i, " : "
  LIRE note
  somme ← somme + note
  SI note > max ALORS max ← note FIN SI
  SI note < min ALORS min ← note FIN SI
FIN POUR
moyenne ← somme / 5
AFFICHER "Moyenne : ", moyenne
SI moyenne >= 10 ALORS
  AFFICHER "Admis"
SINON
  AFFICHER "Ajourné"
FIN SI
AFFICHER "Meilleure note : ", max
AFFICHER "Moins bonne note : ", min

```

Table de traçage (notes [8, 12, 15, 6, 19])

i	note	somme	max	min
0	0	0	0	20
1	8	8	8	8
2	12	20	12	8
3	15	35	15	8
4	6	41	15	6
5	19	60	19	6

Moyenne = $60 / 5 = 12$ | **Admis** | Meilleure : 19 | Moins bonne : 6

Code Python Équivalent

```

max_note = 0
min_note = 20
somme = 0

for i in range(1, 6):
    note = float(input(f"Note {i} : "))
    somme += note
    if note > max_note:
        max_note = note
    if note < min_note:
        min_note = note

moyenne = somme / 5
print("Moyenne :", moyenne)
if moyenne >= 10:
    print("Admis")
else:
    print("Ajourné")
print("Meilleure note :", max_note)
print("Moins bonne note :", min_note)

```

Variante mention : ≥ 16 « Très bien » | ≥ 14 « Bien » | ≥ 12 « Assez bien » | ≥ 10 « Passable » | < 10 « Insuffisant »

Généralisation : remplacer 5 par N saisi par l'utilisateur.

Exercice 5 – Complexité (10 min)

Nonc : Pour chaque algorithme, estimer la complexité ($O(1)$, $O(n)$, $O(n^2)$) :

1. Accéder au 3^{ème} élément d'un tableau
2. Parcourir un tableau de 100 éléments
3. Comparer chaque élément d'un tableau avec tous les autres
4. Lire le premier caractère d'une chaîne
5. Rechercher un nom dans une liste non triée de n noms

Corrigé :

1. $O(1)$ – accès direct par indice
2. $O(n)$ – parcours linéaire
3. $O(n^2)$ – double boucle imbriquée
4. $O(1)$ – accès direct
5. $O(n)$ – dans le pire cas, on parcourt toute la liste

Analyse de taille

Algorithme	Complexité	Opérations ($n=100$)
Accès par indice	$O(1)$	1
Parcours simple	$O(n)$	100
Double boucle	$O(n^2)$	10 000
Boucle triple	$O(n^3)$	1 000 000

QCM d'auto-évaluation

1. Quel est le type de "42" en pseudo-code ?
a) Entier b) Chaîne c) Réel
2. Une structure conditionnelle utilise le mot-clé :
a) POUR b) SI c) TANT QUE
3. Quelle boucle est la plus adaptée pour parcourir un tableau de taille connue ?
a) POUR b) TANT QUE c) RÉPÉTER
4. La complexité d'une double boucle imbriquée (pour i , pour j) est :
a) $O(n)$ b) $O(n^2)$ c) $O(1)$
5. Dans l'algorithme de moyenne, trouver la note maximale nécessite :
a) Un parcours du tableau b) Un tri complet c) Une recherche dichotomique

Différenciation pédagogique

Niveau	Approche	Exercices ciblés
Débutant	Travail guidé pas à pas	Ex2 (table de vérité), Ex3 (tracer $n=5$), Ex4 (tracer notes)

Intermédiaire	Écriture autonome	Ex3 (version TANT QUE), Ex4 (Q2 à N notes)
Avancé	Optimisation, généralisation	Ex4 (Q1 à mentions), Ex5 (Q7 à estimation temporelle)

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Variables et types	20 %	Types incorrects	Types partiellement corrects	Types corrects
Structures conditionnelles	20 %	Conditions incorrectes	Syntaxe partiellement correcte	Syntaxe correcte
Boucles	20 %	Boucles incorrectes	Boucles partiellement correctes	Boucles correctes
Algorithme complet	20 %	Algorithme inexistant	Algorithme partiel	Algorithme complet
Complexité algorithmique	20 %	Ne sait pas calculer	Calcul partiel	Calcul correct et justifié

Note pédagogique ENS

Transposition didactique

L'algorithmique est au cœur des programmes de SNT (Seconde) et de NSI (Première/Terminale). En Seconde, on se limite à lire et modifier des algorithmes simples. En Première NSI, les élèves écrivent leurs premiers algorithmes en Python. L'analogie de la recette de cuisine fonctionne bien : les ingrédients sont les variables, les étapes sont les instructions, et les choix (si la sauce est trop liquide, ajouter de la farine) sont les conditions.

Conseil pour l'enseignant formateur

L'exercice 4 (algorithme complet) est l'exercice le plus important : il mobilise toutes les notions du TD. Demander aux étudiants d'écire d'abord l'algorithme sur papier, puis de le tester mentalement avec des notes exemples (par exemple : 8, 12, 15, 6, 19). Vérifier que chaque étudiant suit l'explication pas à pas (traçage manuel). La complexité (exercice 5) est une notion difficile : l'expliquer visuellement avec des graphiques.

Prolongement

Demander aux étudiants de traduire l'algorithme de l'exercice 4 en Python dans un fichier `moyenne.py`. Ajouter une fonction qui détermine si la moyenne est « Très bien » (≥ 16), « Bien » (≥ 14), « Assez bien » (≥ 12), « Passable » (≥ 10) ou « Insuffisant » (< 10). Tester avec différents jeux de notes.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_FR_06.pdf`

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 06 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- Manipuler les tableaux (parcours, somme, recherche d'extremum)
- Simuler le fonctionnement d'une pile (LIFO) et d'une file (FIFO)
- Choisir la structure de donnÃ©es adaptÃ©e Ã un problÃ¨me concret
- ImplÃ©menter un algorithme de vÃ©rification de parenthÃ©sage avec une pile
- Distinguer les propriÃ©tÃ©s et usages de chaque structure de donnÃ©es

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” Tableaux	15 min	20 min
Exercice 2 â€” Pile	20 min	40 min
Exercice 3 â€” File	20 min	60 min
Exercice 4 â€” Choix de structure	15 min	75 min
Exercice 5 â€” Pile parenthÃ©sage	15 min	90 min
Total	90 min	

Structures de donnÃ©es : Pile et File

```

graph LR
subgraph "Pile (LIFO) â€” Last In, First Out"
direction TB
P1["ðŸŽˆ | 30 â†“ dÃ©piler()"] --> P2["ðŸŽˆ | 20"]
P2 --> P3["ðŸŽˆ | 10"]
P4["empiler(40) â†“ ðŸŽˆ | 40"] --> P1
end

subgraph "File (FIFO) â€” First In, First Out"
direction LR
F1["â€œ... A"] --> F2["â€œ... B"]
F2 --> F3["â€œ... C"]
F4["â€œ... D"] --> F3
end

P4 -->|"empiler"| P1

```

```
P1 -->|"dÃ©piler"| X["x â†± 30"]
F1 -->|"dÃ©filer"| Y["y â†± A"]
F4 -->|"enfiler"| F3
```

```
style P1 fill:#e74c3c,color:#fff
style P2 fill:#e67e22,color:#000
style P3 fill:#4a90d9,color:#fff
style P4 fill:#2ecc71,color:#fff
style F1 fill:#2ecc71,color:#fff
style F2 fill:#4a90d9,color:#fff
style F3 fill:#9b59b6,color:#fff
style F4 fill:#e67e22,color:#000
```

Fonctionnement pas Ã pas â€” Pile (LIFO)

```
flowchart LR
  subgraph "1. Initiale []"
    V1[" "]
  end
  subgraph "2. empiler(10)"
    V2["[10]"]
  end
  subgraph "3. empiler(20)"
    V3["[10, 20]"]
  end
  subgraph "4. empiler(30)"
    V4["[10, 20, 30]"]
  end
  subgraph "5. depiler() â†’ 30"
    V5["[10, 20]"]
  end
  V1 --> V2 --> V3 --> V4 --> V5
```

Fonctionnement pas Ã pas â€” File (FIFO)

```
flowchart LR
  subgraph "1. Initiale []"
    W1[" "]
  end
  subgraph "2. enfiler(A)"
    W2["[A]"]
  end
  subgraph "3. enfiler(B)"
    W3["[A, B]"]
  end
  subgraph "4. defiler() â†’ A"
    W4["[B]"]
  end
  subgraph "5. enfiler(C)"
    W5["[B, C]"]
  end
  W1 --> W2 --> W3 --> W4 --> W5
```

Algorithme de vÃ©rification de parenthÃ©sage

```
flowchart TD
  A[DÃ©but] --> B[Lire l'expression]
  B --> C[Initialiser pile vide]
  C --> D[Pour chaque caractÃ¨re c]
  D --> E{c = '(' ?}
  E -->|Oui| F[Empiler '(']
  F --> G[CaractÃ¨re suivant]
```

```

E -->|Non| H{c = ' ' ?}
H -->|Non| G
H -->|Oui| I{File vide ?}
I -->|Oui| J[Erreur : ) sans (]
I -->|Non| K[DÃ©piler]
K --> G
G --> L[Plus de caractÃ©res ?]
L -->|Non| D
L -->|Oui| M{Pile vide ?}
M -->|Oui| N[Expression correcte â€"]
M -->|Non| O[Erreur : il manque des )]

```

```

style A fill:#4a90d9,color:#fff
style N fill:#2ecc71,color:#fff
style J fill:#e74c3c,color:#fff
style O fill:#e74c3c,color:#fff
style F fill:#e67e22,color:#000
style K fill:#e67e22,color:#000

```

Exercice 1 : Tableaux (15 min)

Ã©noncÃ© : On a le tableau suivant : `notes = [12, 15, 8, 19, 10]`

1. Quelle est la valeur de l'Ã©lÃ©ment Ã l'indice 3 ? (Indices commenÃ§ant Ã 1)
2. Ã©crire un algorithme qui calcule la somme des Ã©lÃ©ments du tableau.
3. Ã©crire un algorithme qui trouve la valeur maximale.

CorrigÃ© :

1. `notes[3] = 8` (les indices commencent Ã 1 en pseudo-code conventionnel)

```

// 2. Somme
somme = 0
POUR i DE 1 Ã 5 FAIRE
    somme = somme + notes[i]
FIN POUR
AFFICHER "Somme : ", somme

```

```

// 3. Maximum
max = notes[1]
POUR i DE 2 Ã 5 FAIRE
    SI notes[i] > max ALORS
        max = notes[i]
    FIN SI
FIN POUR
AFFICHER "Max : ", max

```

Table de traÃ§age : Somme

i	notes[i]	somme
1	12	12
2	15	27
3	8	35
4	19	54
5	10	64

Table de traçage de Maximum

i	notes[i]	max	notes[i] > max ?
1	12	12	(initial)
2	15	15	Oui
3	8	15	Non
4	19	19	Oui
5	10	19	Non

Questions complémentaires

Q1 de Minimum : Écrire l'algorithme pour trouver la valeur minimale.

- **R :** Même principe que max, mais avec < et initialisation à `notes[1]`.

Q2 de Moyenne : Calculer la moyenne des notes.

- **R :** somme / 5 = 64 / 5 = 12,8.

Exercice 2 de Pile (LIFO) (20 min)

Nonc : Simuler l'exécution des opérations suivantes sur une pile initialement vide. Indiquer l'état de la pile après chaque opération.

```
empiler(10)
empiler(20)
empiler(30)
x ← dépiler()
empiler(40)
y ← dépiler()
z ← dépiler()
```

Corrigé :

Opération	Pile après opération	Valeur retournée
Initial	[]	
empiler(10)	[10]	
empiler(20)	[10, 20]	
empiler(30)	[10, 20, 30]	
depiler()	[10, 20]	30
empiler(40)	[10, 20, 40]	
depiler()	[10, 20]	40
depiler()	[10]	20

Question : À quoi correspond cette structure dans un logiciel de traitement de texte ? (Réponse : la fonction Annuler / Ctrl+Z)

Code Python équivalent

```
pile = []
pile.append(10) # empiler(10)
pile.append(20) # empiler(20)
```

```

pile.append(30) # empiler(30)
x = pile.pop() # depiler() â†’ x = 30
pile.append(40) # empiler(40)
y = pile.pop() # depiler() â†’ y = 40
z = pile.pop() # depiler() â†’ z = 20

```

Questions : Quelles sont les valeurs de x, y, z ? Que reste-t-il dans la pile ?

• **R :** x=30, y=40, z=20. Pile : [10].

Exercice 3 – File (FIFO) (20 min)

Nonc : Simuler l'exécution des opérations suivantes sur une file initialement vide.

```

enfiler("A")
enfiler("B")
x ← defiler()
enfiler("C")
y ← defiler()
enfiler("D")
z ← defiler()

```

Corrig :

Opération	File après l'opération	Valeur retournée
Initial	[]	â€
enfiler("A")	["A"]	â€
enfiler("B")	["A", "B"]	â€
defiler()	["B"]	"A"
enfiler("C")	["B", "C"]	â€
defiler()	["C"]	"B"
enfiler("D")	["C", "D"]	â€
defiler()	["D"]	"C"

Question : Dans quel contexte réel utilise-t-on une file d'attente en informatique ? (Réponse : file d'impression, requêtes serveur, buffer vidéo)

Code Python équivalent

```

from collections import deque
file = deque()
file.append("A") # enfiler("A")
file.append("B") # enfiler("B")
x = file.popleft() # defiler() â†’ x = 'A'
file.append("C") # enfiler("C")
y = file.popleft() # defiler() â†’ y = 'B'
file.append("D") # enfiler("D")
z = file.popleft() # defiler() â†’ z = 'C'

```

Question : Quelle différence fondamentale entre pile et file ? Quel élément reste dans la file ?

• **R :** Pile=LIFO, File=FIFO. Il reste ["D"].

Exercice 4 – Choix de structure (15 min)

Question : Quelle structure de données (tableau, pile, file, liste chaînée) est la plus adaptée pour :

1. L'historique de navigation (page précédente)
2. Une liste d'attente au guichet
3. Les notes des étudiants d'une classe (accès rapide par numéro d'étudiant)
4. Une liste de courses où on ajoute/supprime au milieu fréquemment
5. Les appels à une fonction récursive

Correction :

1. **Pile** (LIFO → on remonte dans l'historique)
2. **File** (FIFO → premier arrivé, premier servi)
3. **Tableau** (accès direct par indice = numéro d'étudiant)
4. **Liste chaînée** (insertion/suppression au milieu sans décaler les éléments)
5. **Pile** (pile d'appels → chaque appel ajoute un cadre de pile, le retour d'appel)

Justifications détaillées

Cas	Structure	Pourquoi pas une autre ?
Historique navigation	Pile	Une file donnerait l'ordre inverse (première page visitée en premier)
Liste d'attente	File	Une pile servirait le dernier arrivé en premier (injuste)
Notes des étudiants	Tableau	Accès $O(1)$ par indice ; une liste chaînée nécessiterait un parcours
Liste de courses	Liste chaînée	Un tableau nécessite de décaler les éléments lors d'insertion/suppression
Appels récursifs	Pile	Chaque appel s'empile, le return d'appel → pile d'appels systématique

Exercice 5 → Algorithme avec pile (15 min)

Question : Vérifier un algorithme qui vérifie si une expression parenthésée est correcte, par exemple $(3 + 2) - (5 - 1)$ est correcte mais $(3 + 2 - (5 - 1)$ ne l'est pas.

Principe : utiliser une pile. Quand on lit (on empile, quand on lit) on dépile. Si on dépile sur une pile vide ou si la pile n'est pas vide à la fin, l'expression est incorrecte.

Correction :

```
expression ← "(3 + 2) - (5 - 1)"
pile ← []

POUR i DE 1 À longueur(expression) FAIRE
  car ← expression[i]
  SI car = '(' ALORS
    empiler(pile, '(')
  SINON SI car = ')' ALORS
    SI pile est vide ALORS
      AFFICHER "Erreur : parenthèse fermante sans ouvrante"
      STOP
    SINON
      depiler(pile)
  FIN SI
FIN SI
FIN POUR
```

```

SI pile est vide ALORS
AFFICHER "Expression correcte"
SINON
AFFICHER "Erreur : il manque ", taille(pile), " parenthÃse(s) fermante(s)"
FIN SI

```

Table de traÃçage pour "(3+2)Ã—(5-1)"

CaractÃre	Action	Pile aprÃs
(	Empiler	['(']
3	Ignorer	['(']
+	Ignorer	['(']
2	Ignorer	['(']
)	DÃ©piler	[]
Ã—	Ignorer	[]
(	Empiler	['(']
5	Ignorer	['(']
-	Ignorer	['(']
1	Ignorer	['(']
)	DÃ©piler	[]

RÃ©sultat : Pile vide â†’ expression correcte âœ“

Extension â€” Multi-types de parenthÃses

Adapter l'algorithme pour gÃ©rer (), [], et {} avec concordance des types.

```

SI car = '(' OU car = '[' OU car = '{' ALORS
empiler(pile, car)
SINON SI car = ')' ALORS
SI pile vide OU depiler(pile) â‰  '(' ALORS erreur
SINON SI car = ']' ALORS
SI pile vide OU depiler(pile) â‰ '[' ALORS erreur
SINON SI car = '}' ALORS
SI pile vide OU depiler(pile) â‰ '{' ALORS erreur
FIN SI

```

QCM d'auto-Ã©valuation

1. Une pile suit le principe :

a) FIFO b) LIFO âœ“ c) LILO

2. La structure adaptÃ©e pour une file d'impression est :

a) Une pile b) Une file âœ“ c) Un tableau

3. Dans un tableau, l'accÃs Ã un Ã©lÃ©ment par son indice a une complexitÃ© :

a) $O(1)$ âœ“ b) $O(n)$ c) $O(n^2)$

4. La fonction Â« Annuler Â» (Ctrl+Z) d'un Ã©diteur utilise :

a) Une file b) Une pile c) Un tableau

5. Quelle structure utilise-t-on pour implémenter un algorithme récursif ?

a) File b) Pile c) Liste chaînée

Grille d'évaluation

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Pile LIFO des opérations	20 %	Opérations incorrectes	Opérations partiellement correctes	Opérations correctes
File FIFO des opérations	20 %	Opérations incorrectes	Opérations partiellement correctes	Opérations correctes
Simulation pile	20 %	Simulation inexacte	Simulation partielle	Simulation correcte
Simulation file	20 %	Simulation inexacte	Simulation partielle	Simulation correcte
Choix structure adaptée	20 %	Aucun choix justifié	1-2 choix justifiés	3 choix justifiés

Note pédagogique ENS

Transposition didactique

Les piles et files sont des notions très visuelles et concrètes. En collège/lycée, on peut les introduire avec des objets physiques : une pile d'assiettes (LIFO) et une file d'attente à la cantine (FIFO). Les élèves peuvent jouer le rôle d'opérations (empiler/dépiler, enfiler/défiler) en se plaçant en file indienne. L'algorithme de parenthésage est un classique de NSI Terminale.

Conseil pour l'enseignant formateur

Pour les exercices 2 et 3, faire jouer les simulations au tableau avec des volontaires : un étudiant est la pile, un autre est l'opérateur. Chaque empilement/dépilement est matérialisé par un post-it au tableau. La visualisation physique aide à comprendre la différence entre LIFO et FIFO. L'exercice 4 (choix de structure) est excellent pour la discussion collective : chaque proposition peut être débattue.

Prolongement

Étendre l'exercice 5 pour gérer les trois types de parenthèses : (), [], {}. L'algorithme doit vérifier la concordance : une parenthèse fermante doit correspondre à la dernière ouvrante du même type. Implémenter cet algorithme en Python (`parentheses.py`) avec une classe `Pile`.

Templates : `Template/PG-fr.docx` | `Template/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_FR_07.pdf`

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 07 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- Distinguer les diffÃ©rents types de rÃ©seaux (LAN, WAN, Internet) et leur portÃ©e
- Comprendre le rÃ´le des adresses IP (IPv4/IPv6), du masque et du DNS
- DÃ©crire les Ã©tapes d'une communication client-serveur et le rÃ´le de chaque protocole
- Associer les protocoles rÃ©seau Ã leurs ports par dÃ©faut
- Comparer les mÃ©thodes de partage de fichiers et recommander la plus adaptÃ©e

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” Vocabulaire rÃ©seau	15 min	20 min
Exercice 2 â€” Adresses IP	20 min	40 min
Exercice 3 â€” Client-serveur	15 min	55 min
Exercice 4 â€” Protocoles et ports	20 min	75 min
Exercice 5 â€” Cloud et stockage	15 min	90 min
Total	90 min	

Types de rÃ©seaux et communication client-serveur

```

graph TB
    subgraph "Types de rÃ©seaux"
        LAN["LAN â€” Local Area Network<br/>RÃ©seau local (salle, bÃ¢timent)"]
        WAN["WAN â€” Wide Area Network<br/>RÃ©seau Ã©tendu (ville, pays)"]
        Internet["Internet<br/>RÃ©seau mondial de rÃ©seaux"]
    end

    LAN <--> WAN
    WAN <--> Internet

    subgraph "Communication client-serveur"
        Client[Navigateur<br/>Client] --> DNS[Serveur DNS<br/>Nom â†’ IP]
        Client --> Serveur[Serveur Web]
        Serveur --> Client
    end
  
```

Internet --> Client

```
style LAN fill:#2ecc71,color:#fff
style WAN fill:#e67e22,color:#000
style Internet fill:#4a90d9,color:#fff
style Client fill:#9b59b6,color:#fff
style DNS fill:#e74c3c,color:#fff
style Serveur fill:#e74c3c,color:#fff
```

Protocoles et ports

```
graph LR
  subgraph "Couche Application"
    HTTP["HTTP :80<br/>Navigation non chiffree"]
    HTTPS["HTTPS :443<br/>Navigation chiffree"]
    FTP["FTP :21<br/>Transfert fichiers"]
    SSH["SSH :22<br/>Accès distant sécurisé"]
    SMTP["SMTP :25<br/>Envoi d'emails"]
    IMAP["IMAP :143<br/>Réception d'emails"]
  end
```

```
subgraph "Couche Transport"
  TCP["TCP<br/>Connexion fiable"]
  UDP["UDP<br/>Rapide, non fiable"]
end
```

```
HTTP --> TCP
HTTPS --> TCP
FTP --> TCP
SSH --> TCP
SMTP --> TCP
IMAP --> TCP
```

```
style HTTP fill:#4a90d9,color:#fff
style HTTPS fill:#2ecc71,color:#fff
style FTP fill:#e67e22,color:#000
style SSH fill:#9b59b6,color:#fff
style SMTP fill:#e74c3c,color:#fff
style IMAP fill:#e74c3c,color:#fff
style TCP fill:#4a90d9,color:#fff
style UDP fill:#4a90d9,color:#fff
```

Exercice 1 – Vocabulaire réseau (15 min)

Nonc : Associer chaque terme à sa définition.

Terme	Définition
1. LAN	A. Réseau mondial interconnectant des millions de réseaux
2. WAN	B. Adresse unique d'un appareil sur le réseau
3. Internet	C. Réseau local (bâtiment, salle)
4. Adresse IP	D. Traduit les noms de domaine en adresses IP
5. DNS	E. Réseau étendu (ville, pays)
6. URL	F. Adresse d'une ressource sur le Web

Corrigé : 1→C, 2→E, 3→A, 4→B, 5→D, 6→F

Exercice 2 – Adresses IP (20 min)

À noter :

1. Quelle est la différence entre IPv4 et IPv6 ?
2. À quelle classe appartient l'adresse 192.168.1.1 ?
3. Combien d'adresses IPv4 différentes peut-on coder au maximum ?
4. Combien d'adresses IPv6 ?
5. Que permet la commande `ping 8.8.8.8` ?

Corrigé :

1. IPv4 : 32 bits (4 octets), ~4,3 milliards d'adresses. IPv6 : 128 bits, nombre astronomique d'adresses.
2. 192.168.x.x est une adresse privée de classe C (réseaux locaux).
3. $2^3 \times 2^2 = 4\,294\,967\,296$ (environ 4,3 milliards).
4. $2^{32} = 4\,294\,967\,296$ (340 milliards).
5. `ping 8.8.8.8` teste la connectivité vers le serveur DNS de Google – si ça répond, la connexion Internet fonctionne.

Exercice 3 – Client-serveur (15 min)

À noter : Décrire les étapes de communication quand un utilisateur visite `https://www.ens-mostaganem.dz`.

Pour chaque étape, indiquer si c'est le client (navigateur) ou le serveur qui agit.

1. Saisie de l'URL dans la barre d'adresse
2. Requête DNS pour trouver l'adresse IP
3. Établissement de la connexion TCP
4. Négociation TLS (chiffrement)
5. Envoi de la requête HTTP GET
6. Traitement de la requête par le serveur
7. Envoi de la réponse (page HTML)
8. Affichage de la page dans le navigateur

Corrigé :

1. Client (utilisateur)
2. Client (le navigateur interroge le DNS)
3. Client et serveur (TCP handshake)
4. Client et serveur (TLS handshake)
5. Client
6. Serveur
7. Serveur

8. Client

Exercice 4 – Protocoles et ports (20 min)

À noter : Associer chaque protocole à son port par défaut et à son utilisation.

Protocole	Port	Utilisation
HTTP	22	Transfert de fichiers sécurisés
HTTPS	80	Accès distant sécurisé
FTP	21	Navigation web non chiffrée
SSH	443	Navigation web chiffrée
SMTP	25	Réception d'emails
IMAP	143	Envoi d'emails

Corrigé :

Protocole	Port	Utilisation
HTTP	80	Navigation web non chiffrée
HTTPS	443	Navigation web chiffrée
FTP	21	Transfert de fichiers
SSH	22	Accès distant sécurisé
SMTP	25	Envoi d'emails
IMAP	143	Réception d'emails (synchro multi-appareils)

Exercice 5 – Cloud et stockage (15 min)

À noter : Vous devez partager un dossier de 50 Mo (cours, TD, images) avec votre groupe de TD.

1. Quels sont les avantages et inconvénients de chaque méthode :

- a) Clé USB
- b) Email (pièce jointe)
- c) Google Drive / OneDrive
- d) Moodle

2. Quelle méthode recommandez-vous et pourquoi ?

Corrigé :

Méthode	Avantages	Inconvénients
Clé USB	Pas besoin d'Internet, rapide	Risque perte/virus, uniquement en présentiel
Email	Simple, traçable	Limite pièce jointe (souvent 25 Mo)
Drive/OneDrive	Partage par lien, accessible partout, versioning	Nécessite Internet et compte
Moodle	Intégré à l'université, sécurisé	Moins flexible, interface parfois lourde

Recommandation : Drive ou Moodle pour 50 Mo (l'email est limité à 25 Mo). La clé USB en complément pour le présentiel.

QCM d'auto-évaluation

1. Un réseau LAN s'étend sur :

a) Une ville b) Un bâtiment ou une salle c) Le monde entier

2. IPv4 utilise des adresses sur :

a) 32 bits b) 64 bits c) 128 bits

3. Le protocole HTTPS utilise le port :

a) 80 b) 443 c) 22

4. Le rôle du DNS est de :

a) Chiffrer les communications b) Traduire un nom de domaine en adresse IP c) Gérer les emails

5. Quel protocole utilise-t-on pour envoyer un email ?

a) HTTP b) FTP c) SMTP

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Types de réseaux (LAN/WAN)	15 %	Ne distingue pas les types	1-2 types corrects	3 types corrects
Adressage IP et masque	20 %	Calculs incorrects	Calculs partiellement corrects	Calculs corrects
Modèle client-serveur	20 %	Ne comprend pas le modèle	Compréhension partielle	Compréhension complète
Protocoles et ports	20 %	Aucun protocole identifié	2-3 protocoles	5 protocoles corrects
Cloud et stockage	15 %	Ne connaît pas les services	1-2 services cités	3+ services avec avantages
Schéma réseau	10 %	Schéma inexistant	Schéma partiel	Schéma complet et cohérent

Note pédagogique ENS

Transposition didactique

Les concepts réseau peuvent être introduits en SNT Seconde avec des analogies simples : le réseau local est comme les salles d'une école reliées par des couloirs, Internet est comme le réseau routier mondial. L'adresse IP est comme une adresse postale, le DNS est comme l'annuaire téléphonique. Pour les protocoles, on peut comparer HTTP à une carte postale (texte clair) et HTTPS à une lettre cachetée.

Conseil pour l'enseignant formateur

L'exercice 3 (client-serveur) est central : demander aux étudiants de rejouer la scène en binômes, l'un jouant le client et l'autre le serveur. Chaque étape est verbalisée. Pour l'exercice 4 (ports), faire créer une fiche mémoire plastifiée avec les protocoles/ports à réviser régulièrement. L'exercice 5 (Cloud) est très concret : faire témoigner les étudiants de leurs propres pratiques de partage de fichiers.

Prolongement

Configurer un petit serveur web local avec Python (`python -m http.server 8000`) et y accéder depuis un navigateur. Observer les requêtes avec les outils d'inspecteur (F12 à l'onglet Réseau). Identifier les codes HTTP (200, 404, 500) et comprendre ce qu'ils signifient.

Templates : `Templatte/PG-fr.docx` | `Templatte/Presentation_BELACEL_V3.pptx`

PDF : `TD_Informatique_ENS_1ère_année_FR_08.pdf`

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 08 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- Identifier les trois piliers de la sÃ©curitÃ© (ConfidentialitÃ©, IntÃ©gritÃ©, DisponibilitÃ©)
- Ã‰valuer la robustesse d'un mot de passe et appliquer les bonnes pratiques
- ReconnaÃ®tre les signes d'une tentative de phishing
- Appliquer la rÃ¨gle de sauvegarde 3-2-1
- Identifier les situations conformes ou non au RGPD

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” Triangle CID	15 min	20 min
Exercice 2 â€” Mots de passe	20 min	40 min
Exercice 3 â€” Phishing	20 min	60 min
Exercice 4 â€” Sauvegarde 3-2-1	15 min	75 min
Exercice 5 â€” RGPD	15 min	90 min
Total	90 min	

Triangle CID de la sÃ©curitÃ©

```
graph TB
    subgraph "Triangle CID"
        C[ConfidentialitÃ©<br/>Seules les personnes autorisÃ©es<br/>peuvent accÃ©der aux donnÃ©es]
        I[IntÃ©gritÃ©<br/>Les donnÃ©es n'ont pas Ã©tÃ©<br/>modifiÃ©es sans autorisation]
        D[DisponibilitÃ©<br/>Les donnÃ©es sont accessibles<br/>quand on en a besoin]
    end

    C <--> I
    I <--> D
    D <--> C

    C -. -> Ex1["ðŸ”” Ransomware : fichiers chiffrÃ©s"]
    I -. -> Ex2["â€”i, Note modifiÃ©e sans autorisation"]
    D -. -> Ex3["â€”E Serveur Moodle inaccessible"]

    style C fill:#4a90d9,color:#fff
```



```

style I fill:#2ecc71,color:#fff
style D fill:#e67e22,color:#000
style Ex1 fill:#e74c3c,color:#fff
style Ex2 fill:#e74c3c,color:#fff
style Ex3 fill:#e74c3c,color:#fff

```

D tection de phishing

```

flowchart TD
    A[Email re  su] --> B{Exp diteur<br/>l gitime ?}
    B -->|Oui| C{Urgence<br/>demand  e ?}
    B -->|Non| Phish[  ' PHISHING]
    C -->|Oui| D{Demande<br/>d'identifiants ?}
    C -->|Non| E{URL<br/>suspecte ?}
    D -->|Oui| Phish
    D -->|Non| F{URL raccourcie<br/>bit.ly, tinyurl ?}
    E -->|Oui| Phish
    E -->|Non| Safe[  ... Email l gitime]
    F -->|Oui| Phish
    F -->|Non| Safe

```

```

style A fill:#4a90d9,color:#fff
style Safe fill:#2ecc71,color:#fff
style Phish fill:#e74c3c,color:#fff
style B fill:#e67e22,color:#000
style C fill:#e67e22,color:#000
style D fill:#e67e22,color:#000
style E fill:#e67e22,color:#000
style F fill:#e67e22,color:#000

```

Exercice 1    Triangle CID (15 min)

  nonc   : Identifier le pilier de la s curit   (Confidentialit  , Int grit  , Disponibilit  ) mis en jeu dans chaque situation :

1. Un ransomware chiffre les fichiers d'un enseignant.
2. Un   tudiant modifie sa note dans le fichier Excel de la classe.
3. Le serveur Moodle est inaccessible le jour du d p  t des TD.
4. Un email contenant des mots de passe est intercept  .
5. Un disque dur tombe en panne sans sauvegarde.

Corrig   :

1. Confidentialit   + Disponibilit   (fichiers inaccessibles et chiffres)
2. Int grit   (donn  es modifi  es sans autorisation)
3. Disponibilit   (service indisponible)
4. Confidentialit   (donn  es intercept  es)
5. Disponibilit   (donn  es perdues)

Exercice 2    Mots de passe (20 min)

  nonc   : Classer les mots de passe suivants du plus faible au plus robuste. Justifier.

1. 123456
2. Mot2Passe!
3. ENS.Mostaganem2026!
4. azerty
5. J'aimeLesTartesAuxPommes!
6. password

Corrigé (du plus faible au plus robuste) :

Rang	Mot de passe	Longueur	Problème
6 (faible)	123456	6	Séquentiel, top 1 des mots de passe
5	password	8	Mot courant, dictionnaire
4	azerty	6	Mot du dictionnaire français
3	Mot2Passe!	10	Mot composé avec substitutions prévisibles
2	J'aimeLesTartesAuxPommes!	27	Très long mais phrase complète (attaque par force brute)
1 (fort)	ENS.Mostaganem2026!	22	Long, mélange casse/chiffres/symboles, non

Règle : min 12 caractères, mélange de types, pas de mot du dictionnaire.

Exercice 3 – Phishing (20 min)

Nonc : Analyser l'email suivant et identifier 5 signes d'alerte :

De: support@enseignemrnt-mosta.dz
Objet: URGENT – Votre compte va être fermé

Bonjour,

Votre compte email universitaire sera désactivé dans 48 heures
faute de mise à jour. Cliquez ici pour confirmer vos identifiants :
<http://bit.ly/2XkL9pQ>

Cordialement,
Le service informatique
ENS Mostaganem

Corrigé – Signes d'alerte :

1. **Expéditeur suspect** : enseignemrnt-mosta.dz (faute d'orthographe, pas le domaine officiel)
2. **Urgence** : « URGENT », « 48 heures » – pression psychologique pour éviter la réflexion
3. **Demande d'identifiants** : aucun service légitime ne demande mot de passe par email
4. **URL raccourcie** : bit.ly masque la vraie destination
5. **Formule vague** : « Le service informatique » sans nom précis

Conduite à tenir : Ne pas cliquer. Signaler à l'administration. Vérifier directement sur le site officiel.

Exercice 4 – Sauvegarde 3-2-1 (15 min)

Nonc : Appliquer la règle 3-2-1 pour les données d'un enseignant qui a :

- Ses cours sur son PC portable
- Ses notes sur une clé USB
- Sa thèse sur le disque dur de son bureau

Proposer une stratégie de sauvegarde concrète.

Corrigé :

Copie	Support	Emplacement
Originale	PC portable	Bureau
Copie 1	Disque dur externe	Domicile (dans un tiroir)
Copie 2	Cloud (OneDrive/Google Drive)	En ligne

3 copies (original + 2 sauvegardes), **2 supports différents** (disque dur + cloud), **1 copie hors site** (cloud).

Pour la thèse : en plus, version papier + clé USB dans un lieu différent (chez un collègue).

Exercice 5 – RGD et Éthique (15 min)

À noter : Pour chaque situation, dire si c'est conforme au RGPD ou non, et pourquoi.

1. Un enseignant affiche la liste des notes avec noms et prénoms dans le couloir.
2. Un enseignant utilise son email personnel Gmail pour contacter ses étudiants.
3. Un enseignant supprime les fichiers élèves à la fin de l'année scolaire.
4. Un enseignant filme ses élèves en cours sans les informer.
5. Un enseignant utilise les données des élèves pour une recherche sans les anonymiser.

Corrigé :

1. **Non conforme** car les notes sont des données personnelles, affichage public interdit sans consentement.
2. **Non conforme** car les emails personnels (Gmail, Yahoo) ne garantissent pas la protection des données. Utiliser l'email institutionnel.
3. **Conforme** car le RGPD impose une durée de conservation limitée aux besoins.
4. **Non conforme** car le droit à l'image exige une autorisation écrite préalable (pour les mineurs, autorisation des parents).
5. **Non conforme** car les données doivent être anonymisées avant utilisation dans une recherche.

QCM d'auto-évaluation

1. Le pilier de la sécurité qui garantit que les données n'ont pas été modifiées est :

- a) Confidentialité b) Intégrité c) Disponibilité

2. Un mot de passe robuste doit :

- a) Être court et simple à retenir b) Contenir au moins 12 caractères mélangeant c) Contenir son prénom

3. Un email qui vous demande de cliquer d'urgence pour confirmer votre mot de passe est :

a) Un message l'@gitime du service informatique b) Une tentative de phishing @ c) Un email automatique

4. La r@gle de sauvegarde 3-2-1 signifie :

a) 3 copies, 2 supports, 1 hors site @ b) 3 sauvegardes par jour c) 2 cl@s USB et 1 disque dur

5. Selon le RGPD, les donn@es personnelles doivent @tre :

a) Conserv@es ind@finiment b) Anonymis@es pour la recherche @ c) Partag@es sur les r@seaux sociaux

Grille d'@valuation

Crit@re	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Triade CID	20 %	Ne conna@t pas les 3 piliers	2 piliers sur 3	3 piliers corrects avec exem
Mots de passe robustes	20 %	Ne distingue pas fort/faible	Crit@res partiellement identif@s	Crit@res corrects + outil de
Phishing	20 %	Ne rep@re aucun indice	1-2 indices rep@r@s	3+ indices corrects
S@curisation poste	20 %	Aucune mesure propos@e	1-2 mesures	3+ mesures pertinentes
RGPD et @thique	20 %	Ne conna@t pas le RGPD	Principes partiellement compris	Principes bien compris + ex

Note p@dagogique ENS

Transposition didactique

La s@curit@ num@rique est un th@me du programme de SNT (Seconde) et un enjeu citoyen majeur. Le triangle CID peut @tre enseign@ avec des exemples de la vie quotidienne : la confidentialit@ c'est le journal intime sous cl@, l'int@grit@ c'est un document dont on v@rifie qu'il n'a pas @t@ modifi@, la disponibilit@ c'est pouvoir acc@der @ son cahier quand on en a besoin. Le phishing peut faire l'objet d'un atelier pratique avec de faux emails pr@par@s par l'enseignant.

Conseil pour l'enseignant formateur

L'exercice 2 (mots de passe) est tr@s r@v@lateur : demander aux @tudiants de tester leurs propres mots de passe sur des sites comme @ howsecureismypassword.net @ (en veillant @ ne pas saisir de vrais mots de passe). L'exercice 3 (phishing) peut @tre transform@ en jeu : pr@parer 5 emails (3 vrais l'@gitimes et 2 phishing) et demander aux @tudiants de les trier. L'exercice 5 (RGPD) est crucial pour la formation citoyenne des futurs enseignants.

Prolongement

Organiser un d@fi @ Capture The Flag @ (CTF) adapt@ aux d@butants : les @tudiants doivent identifier des emails de phishing, trouver des mots de passe faibles dans des sc@narios, et proposer des corrections. Utiliser la plateforme @ Root-Me @ ou un CTF maison pr@par@ par l'enseignant avec des @preuves de s@curit@ @l@mentaire.

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_1@re_ann@e_FR_09.pdf

TD Informatique â€” ENS 1^{re} annÃ©e â€” SÃ©ance 09 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1^{re} annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- ImplÃ©menter des algorithmes de recherche (sÃ©quentielle, dichotomique) en Python
- ImplÃ©menter des algorithmes de tri simples (tri par sÃ©lection, tri Ã bulles)
- Manipuler avancÃ©e des listes Python (list comprehension, slicing, filtrage)
- Analyser la complexitÃ© algorithmique d'un code Python
- RÃ©soudre des problÃ©mes algorithmiques en utilisant les structures de donnÃ©es vues en cours

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil et rappels algorithmique	5 min	5 min
Exercice 1 â€” Recherche	20 min	25 min
Exercice 2 â€” Tri par sÃ©lection	20 min	45 min
Exercice 3 â€” Pile et parenthÃ©sage	20 min	65 min
Exercice 4 â€” Listes et filtrage	15 min	80 min
Exercice 5 â€” Tri fusion	10 min	90 min
Total	90 min	

Comparaison des algorithmes de recherche

```

flowchart TB
    subgraph "Recherche sÃ©quentielle"
        direction LR
        S1[DÃ©but] --> S2[Parcourir  
Ã©lÃ©ment par Ã©lÃ©ment]
        S2 --> S3{TrouvÃ© ?}
        S3 -->|Oui| S4[Retourner indice]
        S3 -->|Non| S5[Fin du tableau ?]
        S5 -->|Non| S2
        S5 -->|Oui| S6[Retourner -1]
    end

```

```

subgraph "Recherche dichotomique"
    direction LR
    D1[DÃ©but] --> D2[Tableau triÃ© ?]
    D2 --> D3[Regarder au milieu]
    D3 --> D4{TrouvÃ© ?}
end

```

```

D4 -->|Oui| D5[Retourner indice]
D4 -->|Non| D6{Plus grand<br/>ou plus petit ?}
D6 -->|Plus grand| D7[Chercher Ã droite]
D6 -->|Plus petit| D8[Chercher Ã gauche]
D7 --> D3
D8 --> D3
end

S6 --> Comp[Comparaison<br/>SÃquentielle :  $O(n)$ <br/>Dichotomique :  $O(\log n)$ ]
D5 --> Comp

style S1 fill:#4a90d9,color:#fff
style S4 fill:#2ecc71,color:#fff
style S6 fill:#e74c3c,color:#fff
style D1 fill:#4a90d9,color:#fff
style D5 fill:#2ecc71,color:#fff
style Comp fill:#9b59b6,color:#fff

```

ComplexitÃ des algorithmes de tri

```

graph LR
  subgraph "Algorithmes de tri"
    TS[Tri par sÃlection<br/> $O(n^2)$ ]
    TB[Tri Ã bulles<br/> $O(n^2)$ ]
    TF[Tri fusion<br/> $O(n \log n)$ ]
    TR[Tri rapide<br/> $O(n \log n)$ ]
  end

  subgraph "Performance (n = 1000)"
    TS --> T1["~500 000 comparaisons"]
    TB --> T2["~500 000 comparaisons"]
    TF --> T3["~10 000 comparaisons"]
    TR --> T4["~10 000 comparaisons"]
  end

  TS -. ->| "Simple Ã implÃmenter<br/>Mais lent" | Note1
  TF -. ->| "Plus complexe<br/>Mais rapide" | Note2

  style TS fill:#e74c3c,color:#fff
  style TB fill:#e74c3c,color:#fff
  style TF fill:#2ecc71,color:#fff
  style TR fill:#2ecc71,color:#fff
  style T1 fill:#e67e22,color:#000
  style T2 fill:#e67e22,color:#000
  style T3 fill:#e67e22,color:#000
  style T4 fill:#e67e22,color:#000

```

Exercice 1 â€” Recherche sÃquentielle et dichotomique (20 min)

ÃnoncÃ : CrÃer un script recherche.py qui implÃmente deux algorithmes de recherche :

1. recherche_sequentielle(liste, valeur) â€” parcourt la liste et retourne l'indice ou -1
2. recherche_dichotomique(liste, valeur) â€” recherche dans une liste triÃe (diviser pour rÃgner)

Pour chaque algorithme, compter le nombre de comparaisons effectuÃes et les comparer.

CorrigÃ :

```

def recherche_sequentielle(liste, valeur):
    """Recherche sÃquentielle. Retourne l'indice ou -1."""

```

```

comparaisons = 0
for i, v in enumerate(liste):
    comparaisons += 1
    if v == valeur:
        print(f" S  quentielle : {comparaisons} comparaison(s)")
        return i
print(f" S  quentielle : {comparaisons} comparaison(s)")
return -1

def recherche_dichotomique(liste, valeur):
    """Recherche dichotomique (liste tri  e). Retourne l'indice ou -1."""
    comparaisons = 0
    debut, fin = 0, len(liste) - 1
    while debut <= fin:
        comparaisons += 1
        milieu = (debut + fin) // 2
        if liste[milieu] == valeur:
            print(f" Dichotomique : {comparaisons} comparaison(s)")
            return milieu
        elif liste[milieu] < valeur:
            debut = milieu + 1
        else:
            fin = milieu - 1
    print(f" Dichotomique : {comparaisons} comparaison(s)")
    return -1

# Test
donnees = [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78, 89, 92, 99, 101]
print(f"Liste : {donnees}")
print(f"Recherche de 67 :")
print(f" Indice trouv   : {recherche_sequentielle(donnees, 67)}")
print(f" Indice trouv   : {recherche_dichotomique(donnees, 67)}")

print(f"\nRecherche de 3 (inexistant) :")
print(f" Indice trouv   : {recherche_sequentielle(donnees, 3)}")
print(f" Indice trouv   : {recherche_dichotomique(donnees, 3)}")

```

Question p  dagogique : Sur une liste de 1 000 000   l  ments, combien de comparaisons au maximum pour chaque m  thode ? (*R  ponse : s  quentielle = 1 000 000, dichotomique = $\log_2(1\,000\,000) \approx 20$*)

Exercice 2    Tri par s  lection (20 min)

  nonc   : Cr  er un script `tri_selection.py` qui impl  mente le tri par s  lection. Le programme doit :

1. Impl  menter `tri_selection(liste)` qui trie la liste en place
2. Afficher la liste avant et apr  s le tri
3. Compter le nombre d'  changes et de comparaisons
4. Tester sur une liste g  n  r  e al  atoirement de 10 entiers

Rappel du principe :    chaque it  ration, on cherche le minimum dans la partie non tri  e et on le place en d  but de cette partie.

Corrig   :

```

import random

def tri_selection(liste):
    """Tri par s  lection. Retourne le nombre d'  changes et comparaisons."""

```

```

n = len(liste)
echanges = 0
comparaisons = 0
for i in range(n - 1):
    min_idx = i
    for j in range(i + 1, n):
        comparaisons += 1
        if liste[j] < liste[min_idx]:
            min_idx = j
    if min_idx != i:
        liste[i], liste[min_idx] = liste[min_idx], liste[i]
    echanges += 1
return echanges, comparaisons

# Test
liste = [random.randint(0, 100) for _ in range(10)]
print(f"Liste originale : {liste}")
ech, comp = tri_selection(liste)
print(f"Liste tri  e : {liste}")
print(f"  changes : {ech}, Comparaisons : {comp}")
print(f"Complexit   : O(n   ) avec n = {len(liste)}   t' ~{len(liste)**2 // 2} comparaisons")

```

Exercice 3   " Pile avec validation de parenth  ses (20 min)

  nonc   : Cr  er un script `parentheses.py` qui   tend l'algorithme de validation de parenth  ses vu en cours. Le programme doit :

1. Impl  menter une classe `Pile` avec les m  thodes `empiler()`, `depiler()`, `est_vide()`, `taille()`
2. Utiliser cette classe pour valider des expressions avec `()`, `[]`, `{}`
3. En cas d'erreur, indiquer pr  cis  ment o   se trouve le probl  me (position du caract  re)

Corrig   :

```

class Pile:
    """Impl  mentation d'une pile LIFO."""
    def __init__(self):
        self._elements = []

    def empiler(self, element):
        self._elements.append(element)

    def depiler(self):
        if not self.est_vide():
            return self._elements.pop()
        return None

    def sommet(self):
        if not self.est_vide():
            return self._elements[-1]
        return None

    def est_vide(self):
        return len(self._elements) == 0

    def taille(self):
        return len(self._elements)

    def valider_parentheses(expression):
        """Valide le parenth  sage et retourne (valide, erreur_position, message)."""
        correspondance = {")": "(", "]" : "[", "}" : "{"}

```



```

ouvrantes = "([{"
fermantes = ")]}"
pile = Pile()

for i, char in enumerate(expression):
    if char in ouvrantes:
        pile.empiler(char)
    elif char in fermantes:
        if pile.est_vide():
            return False, i, f"ParenthÃ¨se fermante '{char}' sans ouvrante Ã  la position {i}"
        sommet = pile.depiler()
        if sommet != correspondance[char]:
            return False, i, f"Non concordance : '{sommet}' ouvert mais '{char}' fermÃ© Ã  la position {i}"

if not pile.est_vide():
    return False, len(expression) - 1, f"Il manque {pile.taille()} parenthÃ¨se(s) fermante(s)"

return True, -1, "Expression correcte"

# Tests
expressions = [
    "(3 + 2) Ã [5 - {1 + 2}]",
    "(3 + 2] Ã [5 - {1 + 2})",
    "((1 + 2) Ã 3)",
    "({[]})",
    "((( )))"
]

for expr in expressions:
    valide, pos, msg = valider_parentheses(expr)
    etat = "â€œ" if valide else "â€"
    print(f"{etat} {expr[:50]:50s} â†’ {msg}")

```

Exercice 4 â€” Algorithmes sur les listes (filtrage et transformation) (15 min)

ÃnoncÃ© : CrÃ©er un script `filtrage.py` qui, Ã partir d'une liste de notes alÃ©atoires (20 notes entre 0 et 20) :

1. Calcule la moyenne, la mÃ©diane et l'Ã©cart-type manuellement (sans `statistics` module)
2. Filtre pour obtenir les notes â‰¥ 10 (admis)
3. CrÃ©e une nouvelle liste avec les notes augmentÃ©es de 1 point (bonus, plafonnÃ© Ã 20)
4. Utilise des listes en comprÃ©hension (list comprehension)

CorrigÃ© :

```

import random
import math

notes = [random.randint(0, 20) for _ in range(20)]
print(f"Notes originales : {notes}")

# 1. Statistiques manuelles
moyenne = sum(notes) / len(notes)
notes_triees = sorted(notes)
n = len(notes_triees)
mediane = (notes_triees[n // 2 - 1] + notes_triees[n // 2]) / 2 if n % 2 == 0 else notes_triees[n // 2]
variance = sum((x - moyenne) ** 2 for x in notes) / n
ecart_type = math.sqrt(variance)

```

```

print(f"Moyenne : {moyenne:.2f}")
print(f"Médiane : {mediane:.2f}")
print(f"Écart-type : {ecart_type:.2f}")

# 2. Notes à ¥ 10
admis = [n for n in notes if n >= 10]
print(f"Admis (à ¥10) : {admis} ({len(admis)}/{len(notes)})")

# 3. Bonus plafonné
bonus = [min(n + 1, 20) for n in notes]
print(f"Avec bonus : {bonus}")

```

Exercice 5 – Algorithme de tri fusion (récursif) (10 min)

Nonc : Créer un script `tri_fusion.py` qui implémente le tri fusion (merge sort), un algorithme récursif de complexité $O(n \log n)$.

Principe : diviser la liste en deux, trier chaque moitié récursivement, puis fusionner les deux moitiés triées.

Corrigé :

```

import random

def tri_fusion(liste):
    """Tri fusion récursif."""
    if len(liste) <= 1:
        return liste

    milieu = len(liste) // 2
    gauche = tri_fusion(liste[:milieu])
    droite = tri_fusion(liste[milieu:])

    return fusionner(gauche, droite)

def fusionner(gauche, droite):
    """Fusionne deux listes triées."""
    resultat = []
    i = j = 0
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i += 1
        else:
            resultat.append(droite[j])
            j += 1
    resultat.extend(gauche[i:])
    resultat.extend(droite[j:])
    return resultat

# Test
liste = [random.randint(0, 100) for _ in range(15)]
print(f"Originale : {liste}")
triee = tri_fusion(liste)
print(f"Triée : {triee}")
print(f"Complexité :  $O(n \log n)$  avec  $n = {len(liste)}$ ")

```

QCM d'auto-évaluation

1. La recherche dichotomique nécessite que la liste soit :
a) Non triée b) Triée c) De taille fixe
2. La complexité du tri par sélection est :
a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$
3. En Python, la syntaxe `[x*2 for x in range(5)]` est :
a) Une boucle for b) Une liste en compréhension c) Une fonction lambda
4. Le tri fusion a une complexité de :
a) $O(n^2)$ b) $O(n \log n)$ c) $O(\log n)$
5. Quelle méthode de pile retourne l'élément du dessus sans le retirer ?
a) `empiler()` b) `depiler()` c) `sommet()`

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Recherche séquentielle	15 %	Algorithme incorrect	Algorithme partiel	Algorithme correct
Recherche dichotomique	15 %	Algorithme incorrect	Algorithme partiel	Algorithme correct
Tri par sélection	20 %	Tri incorrect	Tri partiellement correct	Tri correct
Pile et parenthésage	20 %	Algorithme incorrect	Algorithme partiel	Algorithme correct
Tri fusion	20 %	Tri incorrect	Tri partiellement correct	Tri correct
Notion de complexité	10 %	Complexité non identifiée	1-2 complexités correctes	3+ complexités correctes

Note pédagogique ENS

Transposition didactique

Ce TD montre comment les concepts d'algorithmique (recherche, tri) et structures de données (pile) vus en cours s'implémentent concrètement en Python. Ces algorithmes sont au programme de NSI Terminale. La comparaison des complexités ($O(n)$ vs $O(\log n)$ vs $O(n^2)$ vs $O(n \log n)$) est un objectif clé du programme. En classe de Première NSI, on peut commencer par la recherche séquentielle et le tri par sélection avant d'aborder la dichotomie et le tri fusion.

Conseil pour l'enseignant formateur

L'exercice 1 est fondamental : faire exécuter mentalement les deux algorithmes sur un petit jeu de données avant de les coder. Pour l'exercice 2, demander aux étudiants de tracer l'évolution du tableau à chaque itération sur une feuille. L'exercice 3 (parenthésage) est un excellent exemple de l'utilité des piles. L'exercice 5 (tri fusion) introduit la récursivité : la maîtriser est difficile mais cruciale pour la suite du programme.

Prolongement

Proposer un défi : implémenter le tri rapide (quicksort) en Python et comparer ses performances avec le tri fusion sur une liste de 10 000 éléments en utilisant le module `time`. Créer un tableau comparatif des temps d'exécution pour différents algorithmes de tri et différentes tailles de données.

Templates : Templatte/PG-fr.docx | Templatte/Presentation_BELACEL_V3.pptx

PDF : TD_Informatique_ENS_1Ã`re_annÃ©e_FR_10.pdf

TD Informatique â€” ENS 1Ã¨re annÃ©e â€” SÃ©ance 10 (FR)

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS â€” 1Ã¨re annÃ©e â€” DÃ©partement de franÃ§ais

DurÃ©e : 1h30

AnnÃ©e universitaire : 2025-2026

Objectifs pÃ©dagogiques

Ã€ l'issue de cette sÃ©ance de TD, l'Ã©tudiant est capable de :

- ImplÃ©menter une file (FIFO) en Python avec une classe
- RÃ©soudre des problÃªmes algorithmiques utilisant files et piles
- Manipuler des dictionnaires et des structures de donnÃ©es imbriquÃ©es
- ImplÃ©menter un algorithme de parcours en largeur (BFS) simple
- Appliquer les structures de donnÃ©es Ã des problÃªmes concrets

Plan du TD

Phase	DurÃ©e	Total cumulÃ©
Accueil	5 min	5 min
Exercice 1 â€” File d'attente	20 min	25 min
Exercice 2 â€” Expression RPN (pile)	20 min	45 min
Exercice 3 â€” BFS (file + graphe)	20 min	65 min
Exercice 4 â€” Dictionnaires Ã©tudiants	15 min	80 min
Exercice 5 â€” Compression RLE	10 min	90 min
Total	90 min	

Ã©valuation d'expression en notation polonaise inversÃ©e (RPN)

flowchart TD

A[Expression RPN : 5 3 8 + -] --> B[SÃ©parer les Ã©lÃ©ments
['5', '3', '8', '+', '-']]

B --> C[Pile initiale : []]

C --> D1{Lire '5'}

D1 -->|Nombre| E1[Empiler 5]

E1 --> D2{Lire '3'}

D2 -->|Nombre| E2[Empiler 3]

E2 --> D3{Lire '8'}

D3 -->|Nombre| E3[Empiler 8]

E3 --> D4{Lire '+'}

D4 -->|OpÃ©rateur| F4[DÃ©piler 8 et 3]

F4 --> G4[Calculer 3 + 8 = 11]

G4 --> H4[Empiler 11]

H4 --> D5{Lire '-'}

D5 -->|OpÃ©rateur| F5[DÃ©piler 11 et 5]

```
F5 --> G5[Calculer 5 - 11 = -6]
G5 --> H5[Empiler -6]
H5 --> I[R  sultat : -6]
```

```
style A fill:#4a90d9,color:#fff
style I fill:#2ecc71,color:#fff
style E1 fill:#e67e22,color:#000
style E2 fill:#e67e22,color:#000
style E3 fill:#e67e22,color:#000
style G4 fill:#9b59b6,color:#fff
style G5 fill:#9b59b6,color:#fff
style H4 fill:#e67e22,color:#000
style H5 fill:#e67e22,color:#000
```

Parcours en largeur (BFS) d'un graphe

```
graph TD
ENTREE["Entr  e"] --> HALL["Hall"]
HALL --> SALLE1["Salle 1"]
HALL --> CDI["CDI"]
HALL --> BUREAU["Bureau"]
CDI --> SALLE2["Salle 2"]
BUREAU --> SALLE3["Salle 3"]
SALLE1 --> SALLE2
SALLE2 --> SALLE3
```

```
style ENTREE fill:#e74c3c,color:#fff
style HALL fill:#e67e22,color:#000
style SALLE1 fill:#4a90d9,color:#fff
style CDI fill:#4a90d9,color:#fff
style BUREAU fill:#4a90d9,color:#fff
style SALLE2 fill:#2ecc71,color:#fff
style SALLE3 fill:#9b59b6,color:#fff
```

Exercice 1   ” Classe File d'attente (20 min)

  nonc   : Cr  er un script `file_attente.py` qui impl  mente une classe `File` avec :

1. Les m  thodes `enfiler()`, `defiler()`, `tete()`, `est_vide()`, `taille()`
2. Une m  thode `afficher()` qui montre l'  tat de la file
3. Un test avec une simulation de guichet : des clients arrivent et sont servis

Corrig   :

```
from collections import deque

class File:
    """Impl  mentation d'une file FIFO."""
    def __init__(self):
        self._elements = deque()

    def enfiler(self, element):
        self._elements.append(element)

    def defiler(self):
        if not self.est_vide():
            return self._elements.popleft()
        return None

    def tete(self):
```

```

if not self.est_vide():
    return self._elements[0]
return None

def est_vide(self):
    return len(self._elements) == 0

def taille(self):
    return len(self._elements)

def afficher(self):
    print(f"File : {list(self._elements)} (tête : {self.tete()}, {self.taille()} client(s))")

# Simulation de guichet
print("=== Simulation de guichet ===\n")
file = File()

# Arrivée des clients
for nom in ["Ali", "Sara", "Mohamed", "Leila", "Ahmed"]:
    file.enfiler(nom)
print(f"Le client {nom} arrive dans la file.")

print(f"\nÉtat initial :")
file.afficher()

# Service
print("\nService en cours :")
while not file.est_vide():
    client = file.defiler()
    print(f"Le client {client} termine son service.")
    if not file.est_vide():
        print(f"Prochain client : {file.tete()}")

print(f"\nTous les clients ont été servis.")

```

Exercice 2 – Évaluation d'expression postfixe (RPN) (20 min)

Noncible : Créer un script `rpn.py` qui évalue une expression en notation polonaise inversée (NPI/RPN) en utilisant une pile.

Principe : L'opérateur suit ses opérandes. Exemple : $3 \ 4 \ + \ = \ 3 + 4 = 7$

Exemples :

- $3 \ 4 \ + \ = \ 7$
- $5 \ 3 \ 8 \ + \ - \ = \ 5 - (3 + 8) = -6$
- $10 \ 4 \ 2 \ + \ * \ = \ 10 \times (4 + 2) = 60$

Corrigé :

```

def evaluer_rpn(expression):
    """Évalue une expression en notation polonaise inversée."""
    pile = []
    operateurs = {
        "+": lambda a, b: a + b,
        "-": lambda a, b: a - b,
        "*": lambda a, b: a * b,
        "/": lambda a, b: a / b if b != 0 else None
    }

    elements = expression.split()

```

```

for elem in elements:
    if elem in operateurs:
        if len(pile) < 2:
            return "Erreur : pas assez d'opérandes"
        b = pile.pop()
        a = pile.pop()
        resultat = operateurs[elem](a, b)
        if resultat is None:
            return "Erreur : division par zéro"
        pile.append(resultat)
    else:
        try:
            pile.append(float(elem))
        except ValueError:
            return f"Erreur : Élément invalide '{elem}'"

if len(pile) != 1:
    return f"Erreur : {len(pile)} résultats au lieu de 1"
return pile[0]

# Tests
expressions = [
    "3 4 +",
    "5 3 8 + -",
    "10 4 2 + *",
    "15 7 1 1 + - / 3 * 2 1 1 + + -",
    "4 0 /"
]

for expr in expressions:
    resultat = evaluer_rpn(expr)
    print(f"{expr:30s} â†’ {resultat}")

```

Exercice 3 – Parcours en largeur d'un graphe (BFS) (20 min)

À noter : Créer un script `bfs.py` qui implémente un parcours en largeur (Breadth-First Search) en utilisant une file. Le graphe représente le plan d'une école : chaque pièce est reliée à d'autres par des couloirs.

```

graph TD
    ENTREE --> HALL
    HALL --> SALLE1
    HALL --> CDI
    HALL --> BUREAU
    CDI --> SALLE2
    BUREAU --> SALLE3
    SALLE1 --> SALLE2
    SALLE2 --> SALLE3

```

Corrigé :

```

from collections import deque

def bfs(graphe, depart):
    """Parcours en largeur à partir d'un nœud."""
    visites = set()
    file = deque()
    ordre = []

    file.append(depart)
    visites.add(depart)

```



```

print(f"Parcours en largeur depuis '{depart}' :")

while file:
    courant = file.popleft()
    ordre.append(courant)
    print(f" Visite : {courant}")

    for voisin in sorted(graphe[courant]):
        if voisin not in visites:
            visites.add(voisin)
            file.append(voisin)

    return ordre

# Graphe : plan de l'école
ecole = {
    "Entrée": ["Hall"],
    "Hall": ["Entrée", "Salle1", "CDI", "Bureau"],
    "Salle1": ["Hall", "Salle2"],
    "Salle2": ["CDI", "Salle1", "Salle3"],
    "Salle3": ["Bureau", "Salle2"],
    "CDI": ["Hall", "Salle2"],
    "Bureau": ["Hall", "Salle3"]
}

ordre_visite = bfs(ecole, "Entrée")
print(f"\nOrdre de visite : {' '.join(ordre_visite)}")

# Question : quelle est la distance (nombre de portes) entre l'entrée et la salle 3 ?
# Réponse : Entrée → Hall → Bureau → Salle3 (3 portes)

```

Exercice 4 – Dictionnaire d'Étudiants avec recherche multicritère (15 min)

Nonc : Créer un script `etudiants.py` qui gère une base de données d'Étudiants (liste de dictionnaires). Le programme :

1. Crée une liste d'au moins 5 Étudiants avec : nom, notes (liste), groupe, filière
2. Calcule la moyenne de chaque Étudiant
3. Propose des fonctions de recherche par groupe, par filière, par moyenne
4. Trie les Étudiants par moyenne d'croissante

Corrigé :

```

def creer_etudiant(nom, groupe, filiere, notes):
    """Crée un dictionnaire Étudiant."""
    moyenne = sum(notes) / len(notes) if notes else 0
    return {
        "nom": nom,
        "groupe": groupe,
        "filiere": filiere,
        "notes": notes,
        "moyenne": round(moyenne, 2)
    }

def rechercher_par_groupe(etudiants, groupe):
    """Recherche les Étudiants d'un groupe."""
    return [e for e in etudiants if e["groupe"] == groupe]

def rechercher_par_filiere(etudiants, filiere):
    """Recherche les Étudiants d'une filière."""

```

```

return [e for e in etudiants if e["filiere"] == filiere]

def rechercher_par_moyenne(etudiants, seuil):
    """Recherche les Ã©tudiants avec moyenne â‰¥ seuil."""
    return [e for e in etudiants if e["moyenne"] >= seuil]

def trier_par_moyenne(etudiants):
    """Trie par moyenne dÃ©croissante."""
    return sorted(etudiants, key=lambda e: e["moyenne"], reverse=True)

# CrÃ©ation de la base
etudiants = [
    creer_etudiant("Ali", "G1", "FranÃ§ais", [15, 12, 18]),
    creer_etudiant("Sara", "G1", "FranÃ§ais", [18, 16, 17]),
    creer_etudiant("Mohamed", "G2", "Anglais", [10, 11, 9]),
    creer_etudiant("Leila", "G2", "FranÃ§ais", [16, 14, 19]),
    creer_etudiant("Ahmed", "G1", "Anglais", [8, 10, 12]),
    creer_etudiant("Fatima", "G2", "Anglais", [14, 15, 13])
]

print("=== Tous les Ã©tudiants ===")
for e in etudiants:
    print(f"{e['nom']:<10} {e['groupe']:<4} {e['filiere']:<10} Notes : {e['notes']} Moy : {e['moyenne']}")

print(f"\n=== Groupe G1 ===")
for e in rechercher_par_groupe(etudiants, "G1"):
    print(f" {e['nom']} â€” {e['filiere']} â€” Moy : {e['moyenne']}")

print(f"\n=== Moyenne â‰¥ 14 ===")
for e in rechercher_par_moyenne(etudiants, 14):
    print(f" {e['nom']} â€” {e['moyenne']}")

print(f"\n=== Classement gÃ©nÃ©ral ===")
for i, e in enumerate(trier_par_moyenne(etudiants), 1):
    print(f"{i:2d}. {e['nom']:<10} {e['moyenne']:.2f}")

```

Exercice 5 â€” Compression par dictionnaire (run-length encoding) (10 min)

Ã‰noncÃ© : CrÃ©er un script `rle.py` qui implÃ©mente un algorithme simple de compression :

- Le codage RLE (Run-Length Encoding) remplace les rÃ©pÃ©titions par (valeur, nombre)
- Exemple : AAABBBCCDAA â†’ [('A', 3), ('B', 3), ('C', 2), ('D', 1), ('A', 2)]
- ImplÃ©menter `compresser(chaine)` et `decompresser(donnees)`

CorrigÃ© :

```

def compresser(chaine):
    """Comprime une chaÃªne avec RLE."""
    if not chaine:
        return []

    resultat = []
    courant = chaine[0]
    compteur = 1

    for c in chaine[1:]:
        if c == courant:
            compteur += 1
        else:
            resultat.append((courant, compteur))
            courant = c
            compteur = 1
    resultat.append((courant, compteur))

```

```

courant = c
compteur = 1

resultat.append((courant, compteur))
return resultat

def decompresser(donnees):
    """Décompresser des données RLE."""
    return "".join(c * n for c, n in donnees)

# Tests
exemple = "AAABBBCCDAA"
print(f"Originale : {exemple}")
compresse = compresser(exemple)
print(f"Compressée : {compresse}")
print(f"Taux : {len(str(compresse)) / len(exemple) * 100:.1f}% de la taille originale")
decompresse = decompresser(compresse)
print(f"Décompressée : {decompresse}")
print(f"Identique ? {exemple == decompresse}")

# Autre test
texte = "WWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWBWWWWWWWWWWB"
comp = compresser(texte)
print(f"\nTexte original : {len(texte)} caractères")
print(f"Texte compressée : {len(comp)} paires")
print(f"Taux de compression : {len(comp) / len(texte) * 100:.1f}%")

```

QCM d'auto-évaluation

1. Une file (FIFO) utilise le principe :

- a) Dernier entré, premier sorti b) Premier entré, premier sorti c) Accès aléatoire

2. En notation polonaise inversée, l'expression $5 \ 3 \ -$ vaut :

- a) 2 b) -2 c) 8

3. Le parcours en largeur (BFS) utilise une structure de :

- a) Pile b) File c) Tableau

4. En Python, un dictionnaire est une structure de données qui :

- a) Stocke des éléments indexés par des entiers b) Associe des clés à des valeurs c) Stocke des éléments dans l'ordre d'insertion

5. Le codage RLE (Run-Length Encoding) est utile pour compresser :

- a) Des données avec beaucoup de répétitions b) Des données aléatoires c) Des fichiers déjà compressés

Grille d'évaluation

Critère	Poids	Non acquis (0)	Partiel (0.5)	Acquis (1)
Classe Pile/File	20 %	Implémentation absente	Implémentation partielle	Implémentation correcte
Notation polonaise inversée	20 %	Algorithme incorrect	Algorithme partiel	Algorithme correct
Parcours en largeur	20 %	Parcours incorrect	Parcours partiel	Parcours correct
Compression RLE	20 %	Compression incorrecte	Compression partielle	Compression correcte

Interface utilisateur	20 %	Pas d'interface	Interface partielle	Interface fonctionnelle
-----------------------	------	-----------------	---------------------	-------------------------

Note pédagogique ENS

Transposition didactique

Ce TD de synthèse mobilise toutes les compétences acquises : classes, piles, files, dictionnaires, algorithmes. La notation polonaise inversée (exercice 2) est un excellent exemple de l'utilité des piles en informatique. Le parcours en largeur (exercice 3) introduit les graphes, concept fondamental pour l'enseignement de NSI Terminale. La compression RLE (exercice 5) est intuitive et visuelle : les élèves comprennent vite le principe.

Conseil pour l'enseignant formateur

L'exercice 2 (RPN) est souvent source de confusion car les élèves doivent lire l'expression différemment. Faire calculer à la main plusieurs expressions RPN au tableau avant de coder. Pour l'exercice 3 (BFS), utiliser un grand papier ou le tableau pour simuler physiquement le parcours : placer des post-its représentant les salles et les relier par des fils, puis déplacer un pion. L'exercice 5 (RLE) est ludique : demander aux étudiants de trouver des exemples de répétitions dans leur propre nom ou dans des phrases.

Prolongement

Proposer un mini-projet de synthèse : créer un jeu du pendu en Python qui utilise une classe `MotSecret` avec les notions de dictionnaire, de liste et d'algorithmes de recherche vues dans ce TD. Le jeu doit tirer un mot aléatoire, masquer les lettres, et permettre à l'utilisateur de proposer des lettres jusqu'à ce qu'il ait trouvé ou perdu.